

Software Requirements Specification (SRS)

USO Enterprise Platform (UEP)

Version: 1.0 Draft

Author: Solution Architecture Team

Status: In Progress

Chapter 1 — Introduction

1.1 Purpose

The purpose of this document is to define the complete functional and non-functional requirements of the **USO Enterprise Platform (UEP)**.

The platform is designed to replace multiple disconnected Excel trackers, manual email communications, and decentralized approval processes with a centralized web-based system that manages the complete lifecycle of USO project execution.

The system will provide a single source of truth for:

- Drive Test activities
- Contractor management
- Assignment management
- Acceptance workflow
- ICT approvals
- CRA approvals
- Executive dashboards
- Project reporting
- Future operational modules

This document serves as the baseline specification for software development, testing, deployment, maintenance, and future enhancement.

1.2 Background

The current operational process relies heavily on manual activities performed across multiple Excel spreadsheets owned by different stakeholders.

These spreadsheets are exchanged through email and updated independently by:

- Project Support Office (PSO)
- Drive Test Contractors
- ICT Coordinators
- CRA Coordinators
- Regional Managers
- Project Managers

The lack of a centralized platform creates significant operational challenges.

Examples include:

- duplicate data
- inconsistent status
- manual lookup operations
- difficult reporting
- poor traceability
- delayed decision making
- lack of ownership visibility

As the project has been running for several years and manages approximately 500 new sites annually, the existing approach is no longer sustainable.

1.3 Business Problem

The current business process suffers from several operational inefficiencies.

Manual Data Synchronization

Acceptance information is received from multiple independent organizations.

Each organization delivers data using different Excel formats.

Project coordinators manually perform dozens of lookup operations before updating the master tracker.

Multiple Sources of Truth

Different teams maintain independent trackers.

Examples include

- CPM Master File
- Drive Test Tracker
- ICT Tracker
- CRA Tracker
- Contractor Progress Files

These files frequently become inconsistent.

Lack of Process Visibility

Project Managers cannot immediately answer questions such as

- Which contractor is delayed?
 - Which region has the highest backlog?
 - Which villages are waiting for ICT?
 - Which sites are blocked because of temporary power?
 - Which approvals are missing?
-

Reporting Complexity

Generating management reports requires manual Excel processing that may take several hours or even days.

No Ownership Tracking

Responsibilities across different teams are unclear.

The organization lacks a centralized Action Center identifying pending activities for each user.

1.4 Business Objectives

The primary objective of the platform is to establish a centralized operational system that manages the entire lifecycle of every USO Work Item.

Specific objectives include:

BO-01

Eliminate Excel as the operational working environment.

BO-02

Centralize all operational data into a single database.

BO-03

Provide real-time dashboards for all stakeholders.

BO-04

Reduce manual reporting effort.

BO-05

Improve transparency of responsibilities.

BO-06

Track every operational activity using complete audit history.

BO-07

Automate workflow progression wherever business rules allow.

BO-08

Support future business modules without redesigning the architecture.

1.5 Project Scope

The initial release shall support the following business domains.

Included

Drive Test Management

Assignment Management

Health Check

ICT Acceptance

CRA Acceptance

Letter Tracking

Contractor Management

Executive Dashboard

Regional Dashboard

Coordinator Dashboard

Reports

Notification Center

Action Center

Administration

Role Management

Master Data Import

History Tracking

Audit Log

Future Releases

Battery Tracking

Power Tracking

Budget Management

Invoice Management

AMP Integration

Network KPI Integration

API Integration with CPM

AI Decision Support

Predictive Analytics

1.6 Out of Scope

The following functions are explicitly excluded from Version 1.

Route Planning

Drive Test Report Storage

Network Performance Optimization

Radio Planning

Physical File Management

Government Portal Integration

Financial ERP Integration

1.7 Stakeholders

Stakeholder	Responsibility
Admin	System Administration
Project Manager	Project Governance
Coordinator	Acceptance & DT Coordination
Contractor	DT Execution
Regional Manager	Regional Follow-up
Viewer	Read-only Access
Executive Management Portfolio Monitoring	

1.8 Definitions

Term	Definition
Site	Physical telecom location
Village	Coverage area served by a Site
Work Item	Unique execution unit identified by (Site ID + Site Type)
Acceptance	Government approval process
ICT	Ministry ICT Organization
CRA	Communications Regulatory Authority
DT	Drive Test
CPM	Master source system for project deployment

Term	Definition
Assignment	Official notification assigning a Work Item
Health Check	Contractor validation before DT
Approval	Final government confirmation
Rejection	Government refusal requiring corrective action

1.9 Design Principles

The platform shall be developed according to the following architectural principles:

Single Source of Truth

Only one database stores operational information.

Database-Driven Workflow

Workflow progression is determined by database state rather than user assumptions.

Event-Based Processing

Business actions generate events that update workflow status automatically.

Auditability

Every change performed by every user shall be permanently recorded.

Scalability

The architecture shall support future modules without redesign.

Security

Role-Based Access Control (RBAC) shall govern every operation.

Extensibility

Business logic shall remain independent from presentation and reporting.

1.10 Success Criteria

The project shall be considered successful when:

- Excel is no longer required for daily operations.
 - Every stakeholder uses the platform as the primary operational system.
 - Management dashboards are generated in real time.
 - Approval tracking becomes fully transparent.
 - Responsibility for every pending activity is clearly identified.
 - Historical changes are preserved.
 - The system supports future operational modules without architectural modification.
-

End of Chapter 1

Next Chapter: Chapter 2 — Business Process & Business Rules.

This is where we'll formalize the complete lifecycle of a Work Item, including CPM import, Health Check, Assignment, Drive Test, Acceptance, ICT/CRA approvals, rejection loops, notifications, ownership transitions, and automation rules. This chapter will define the business engine that the backend will implement.

Software Requirements Specification (SRS)

Chapter 2 – Business Process & Business Rules

2.1 Overview

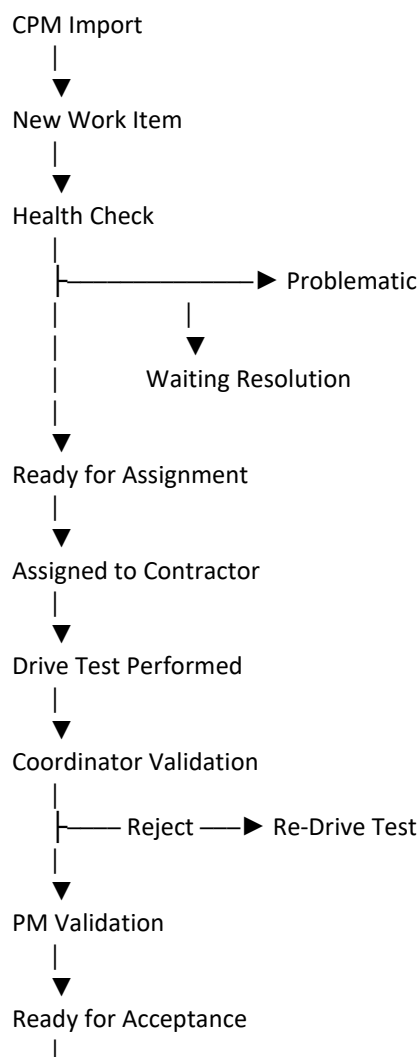
The USO Enterprise Platform manages the complete lifecycle of a **Work Item** from the moment it appears in the CPM master file until all required regulatory approvals are completed.

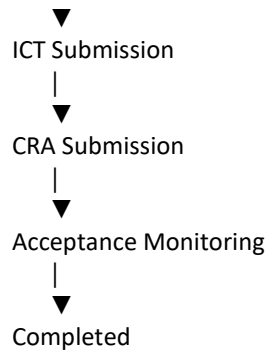
The platform replaces multiple independent Excel trackers with a single workflow engine.

A Work Item progresses through predefined business states, each owned by specific user roles.

2.2 Business Lifecycle

A Work Item shall follow the lifecycle below.





2.3 Master Data Source

The CPM file is the **only master source** for infrastructure information.

The CPM file owns:

- Site ID
- Village ID
- Province
- Site Type
- Requested Technology
- Assignment Date (Official Assignment)
- On-Air Date

No user may manually create a Work Item.

All Work Items originate from CPM.

2.4 CPM Synchronization Rules

The system shall compare every imported CPM file against the database.

For each incoming record:

New Record

Create a new Work Item.

Existing Record (No Change)

Ignore.

Existing Record (Changed)

Create a Change Request.

Notify Project Manager.

PM may choose:

- Accept Change
- Ignore Change
- Archive Existing Version

All decisions shall be stored in history.

2.5 Work Item Definition

A Work Item represents one executable deployment.

Primary Business Key

Site ID + Site Type

Examples

100234 + New Site

100234 + Add Technology

Each Work Item has only one lifecycle.

2.6 Village Mapping

Each Work Item may contain one or more Villages.

Example

Work Item

↓

Village A

Village B

Village C

Villages are imported exclusively from CPM.

Village additions or removals require PM approval.

2.7 Health Check

Health Check is performed by the assigned contractor.

Purpose:

Determine whether the site is technically ready for Drive Test.

Possible Results

Ready

Problematic

Problematic sites require a category.

2.8 Problem Categories

Initial categories

- Temporary Power
- Material Shortage
- MS Responsibility
- On-site Issue

The system shall allow administrators to add new categories without software modification.

2.9 Assignment Process

Two assignment types exist.

Official Assignment

Owner:

ICT

Stored only for reporting.

Cannot be edited.

Contractor Assignment

Owner:

Project Manager

Assigns Work Item to one contractor.

Assignment records include:

Assignment Date

Assigned By

Contractor

Remarks

History

2.10 Assignment Rules

One active contractor per Work Item.

If reassigned:

Previous assignment becomes inactive.

History remains permanently stored.

2.11 Drive Test

The contractor performs one Drive Test for the Work Item.

Contractor updates

Drive Test Date

Status

Comment

Report Link

2.12 Drive Test Validation

Coordinator reviews submitted DT.

Possible outcomes

Approved

Rejected

Returned

If rejected

Workflow returns to

Drive Test.

2.13 PM Validation

Coordinator approval alone is insufficient.

PM must approve.

Only then

Drive Test Status = Completed.

2.14 Acceptance Scope

Acceptance is managed per

Site ID + Village ID

not per Work Item.

This distinction is fundamental.

One Work Item

↓

Many Villages

↓

Each Village

↓

Independent Acceptance

2.15 Acceptance Authorities

Acceptance is performed independently by

ICT Province

ICT Tehran

CRA Region

CRA Tehran

Each authority may issue different official letters.

2.16 Technology Approval

Each Village stores approval for

2G

3G

4G

Each technology has independent status.

Possible values

Approved

Rejected

Pending

Commitment

2.17 Village Status

Village Overall Status

Not Submitted

Waiting ICT

Waiting CRA

Partially Approved

Fully Approved

Rejected

2.18 Site Completion Rule

A Work Item becomes Fully Accepted only when

ALL villages

AND

ALL requested technologies

have reached

Approved.

2.19 Letter Registration

Each approval letter stores

Letter Number

Letter Date

Authority

Province / Region

PDF Attachment (optional)

Comment

One letter may approve

one

or

many villages.

2.20 Acceptance Rejection

Rejected villages shall remain visible.

Coordinator attempts resubmission using existing DT.

If unsuccessful

New DT is required.

History shall preserve every attempt.

2.21 Notification Rules

System notifications shall be generated automatically for

New CPM Changes

Assignment Required

Health Check Completed

DT Submitted

DT Rejected

PM Approval Required

Village Rejected

Village Approved

ICT Waiting

CRA Waiting

Contractor Reassignment

CPM Conflict

2.22 Action Center

Every logged-in user shall immediately see

"My Pending Actions"

Examples

Coordinator

23 DT waiting validation

54 ICT letters pending

12 CRA letters pending

Contractor

15 Assigned

6 Ready

3 Blocked

9 Waiting Validation

PM

4 CPM changes

16 PM approvals

9 Assignment requests

5 DT conflicts

2.23 Audit Rules

Every modification shall store

Timestamp

User

Previous Value

New Value

Reason

Module

IP Address (optional)

History cannot be edited.

2.24 Soft Delete

Operational records shall never be physically deleted.

Soft Delete shall be used.

Deleted records remain recoverable.

2.25 Archive

Completed historical versions may be archived.

Archived data remains searchable.

Archived data is excluded from operational dashboards.

2.26 Business KPIs

The platform shall automatically calculate:

- Total Sites
 - Total Villages
 - On-Air Sites
 - Ready for DT
 - Assigned
 - DT Completed
 - DT Remaining
 - DT Blocked
 - ICT Approved
 - CRA Approved
 - Fully Accepted
 - Partial Acceptance
 - Contractor Productivity
 - Province Completion
 - Region Completion
 - Monthly DT
 - Yearly DT
 - Monthly Acceptance
 - Yearly Acceptance
 - Block Categories
 - Aging Analysis
 - Backlog Trend
-

2.27 Business Automation

The system shall automatically determine:

Ready for DT

Ready for Submission

Waiting ICT

Waiting CRA

Acceptance Complete

PM Action Required

Coordinator Action Required

Contractor Action Required

CPM Conflict

Dashboard Counters

Notifications

No manual status update shall be required where the system can derive the state automatically.

2.28 Future Extension Rules

The architecture shall support future modules without modifying existing business logic.

Future modules include:

- Budget Management
- Battery Tracking
- Permanent Power Tracking
- Invoice Management
- AMP Integration
- API Integration
- AI Analytics
- Executive KPI Engine

Each future module shall reference existing entities rather than duplicate operational data.

End of Chapter 2

Architect Review (Important)

This chapter already defines about **70% of the backend behavior**. However, before we move to Chapter 3 (Domain Model), I noticed **three business gaps** that we have not fully specified. They are critical because they affect the database schema and workflow engine:

1. **Work Item Status Engine** – We currently have many statuses (New, Ready, Assigned, DT Done, etc.). I recommend making all statuses **system-derived** instead of manually editable to prevent inconsistent data.
2. **Acceptance Submission** – We know ICT and CRA submissions happen, but we haven't explicitly defined whether the system tracks the **submission package** itself (who submitted, when, which villages, package ID) or only the approval result. I strongly recommend introducing an AcceptanceSubmission entity.
3. **Action Center Engine** – Instead of hardcoding pending actions, I recommend implementing a **rule engine** that generates tasks dynamically based on workflow conditions. This will make future modules (Budget, Battery, Power, Invoice) automatically integrate into the same Action Center without redesign.

These are architectural improvements that will make the platform significantly more scalable and maintainable. I recommend we adopt them before freezing the SRS.

Software Requirements Specification (SRS)

Chapter 3 — Domain Model

3.1 Introduction

The USO Enterprise Platform is designed according to **Domain-Driven Design (DDD)** principles.

Instead of organizing the software around database tables or UI screens, the system is organized around **business domains**.

Each domain encapsulates its own business rules, entities, services, workflows, and history.

The objective is to ensure that future modules (Budget, Battery, Power, Invoice, AMP, AI, etc.) can be added without redesigning the existing architecture.

3.2 High-Level Domain Model

USO Enterprise Platform

|

└─ Master Data Domain

└─ Work Item Domain

└─ Assignment Domain

└─ Contractor Domain

└─ Health Check Domain

└─ Drive Test Domain

└─ Acceptance Domain

└─ Letter Domain

└─ Dashboard Domain

└─ Reporting Domain

└─ Notification Domain

└─ Action Center Domain

└─ Administration Domain

└─ Audit Domain

3.3 Master Data Domain

Purpose

Maintains all reference information imported from CPM.

Owner

Project Manager

Main Entities

Province

Village

Site

Technology

Site Type

Region

Contractor

Problem Category

Master Data cannot be manually created except by Admin.

Most records originate from CPM.

3.4 Work Item Domain

The Work Item is the core Aggregate Root of the entire platform.

Everything revolves around Work Items.

Primary Key

SiteID + SiteType

Contains

- Requested Technology
- Assignment Information

- DT Status
- Acceptance Status
- Contractor
- Battery Status (Future)
- Budget Status (Future)

Every other operational entity references Work Item.

3.5 Village Domain

Village belongs to a Work Item.

Relationship

One Work Item

↓

Many Villages

Each Village owns

ICT Status

CRA Status

Technology Approval

Letters

History

3.6 Assignment Domain

Responsible for contractor allocation.

Contains

Official Assignment



Contractor Assignment



Assignment History



Reassignment

Assignment history is immutable.

3.7 Contractor Domain

Maintains contractor information.

Includes

Company

Users

Assigned Work Items

Performance

Monthly Progress

Blocked Sites

KPIs

3.8 Health Check Domain

Purpose

Determine DT readiness.

Possible Results

Ready

Blocked

Blocked requires

Problem Category

Comment

History

Owner

Contractor

3.9 Drive Test Domain

One Work Item



One Active Drive Test



Many Historical Drive Tests

The latest approved DT is the active one.

Rejected DTs remain in history.

Drive Test contains

Execution Date

Report Link

Status

Coordinator Review

PM Review

Remarks

3.10 Acceptance Domain

Acceptance belongs to Village.

Relationship

Work Item

↓

Village

↓

Acceptance

Each Acceptance contains

ICT

CRA

Technology Status

Overall Status

History

3.11 Technology Approval Domain

Every Village contains

2G

3G

4G

Each technology has

Approval Status

Approval Date

Comment

Commitment

Technology approvals are independent.

3.12 Letter Domain

Letters are independent entities.

One Letter

↓

Many Villages

Stores

Letter Number

Authority

Province

Issue Date

PDF

Comment

History

3.13 Dashboard Domain

Dashboard never stores data.

Dashboard reads data.

Everything is calculated dynamically.

No duplicated statistics.

3.14 Reporting Domain

Reports are generated dynamically.

Supported Outputs

Excel

PDF

CSV (Future)

Power BI API (Future)

3.15 Notification Domain

Notifications are generated by Events.

Examples

Assignment Created

DT Completed

DT Rejected

ICT Approved

CRA Approved

CPM Conflict

PM Decision Required

Coordinator Action Required

Notifications are never manually created.

3.16 Action Center Domain

Action Center is not a table.

It is a Rule Engine.

Rules generate pending tasks.

Example

IF

DT Approved

AND

PM Approval Missing

↓

Generate

PM Task

Example

ICT Reject

↓

Coordinator Task



Notify Contractor

Future modules automatically plug into Action Center.

3.17 Administration Domain

Responsible for

Users

Roles

Permissions

Reference Data

System Settings

Imports

Exports

Audit

3.18 Audit Domain

Every operational change creates

Audit Event

Audit Events are append-only.

Never editable.

Stores

Timestamp

User

Module

Old Value

New Value

Reason

3.19 Aggregate Roots

To avoid circular dependencies, only the following entities act as Aggregate Roots.

Work Item

Contractor

User

Letter

Notification

Everything else belongs to one Aggregate.

3.20 Domain Events

The system is event-driven.

Examples

CPMImported

AssignmentCreated

AssignmentChanged

HealthCheckCompleted

DriveTestSubmitted

DriveTestRejected

DriveTestApproved

ICTSubmitted

ICTApproved

ICTRejected

CRASubmitted

CRAApproved

CRARejected

LetterRegistered

PMApproved

CPMConflictDetected

Each Event automatically updates:

- Dashboard
- Reports
- Notifications
- Action Center
- Audit Log

3.21 Domain Services

Some business logic belongs to services instead of entities.

Examples

Assignment Service

Drive Test Service

Acceptance Service

Dashboard Service

Reporting Service

Notification Service

Workflow Service

CPM Import Service

Audit Service

Services encapsulate complex workflows and orchestration.

3.22 Domain Invariants

The system shall always enforce the following invariants:

1. A Work Item cannot exist without a Site.
 2. A Village cannot exist without a Work Item.
 3. A Contractor Assignment must reference exactly one active Contractor.
 4. A Work Item may have only one active Contractor Assignment.
 5. A Work Item may have only one active Drive Test.
 6. Historical Drive Tests shall never be modified.
 7. A Village may have multiple Letter references over time.
 8. Acceptance status is always derived from ICT and CRA decisions.
 9. Dashboard values are always calculated, never stored.
 10. History records are immutable.
 11. CPM remains the master source for infrastructure data.
 12. No operational record shall be physically deleted.
-

3.23 Future Domains

The architecture reserves dedicated domains for future expansion.

Budget Domain

Battery Domain

Power Domain

Invoice Domain

End of Chapter 3

Solution Architect Review

At this point we have frozen the **business architecture**. The next chapter is the one that will have the biggest impact on implementation:

Chapter 4 — Entity Catalog & Data Dictionary

This chapter will define **every database table**, including:

- Columns
- Data types
- Primary Keys
- Foreign Keys
- Constraints
- Indexes
- Validation rules
- Relationships
- Default values
- Enumerations
- Calculated fields

I expect this chapter to contain **40–60 entities** and become the blueprint for the PostgreSQL database. Once it is complete, the backend can be implemented almost directly from the specification with minimal interpretation.

Software Requirements Specification (SRS)

Chapter 4 — Entity Catalog & Data Dictionary

Version: Draft 1.0

4.1 Introduction

The Entity Catalog defines every persistent business object within the USO Enterprise Platform.

Each entity represents a single business concept and is responsible for storing operational information throughout the system lifecycle.

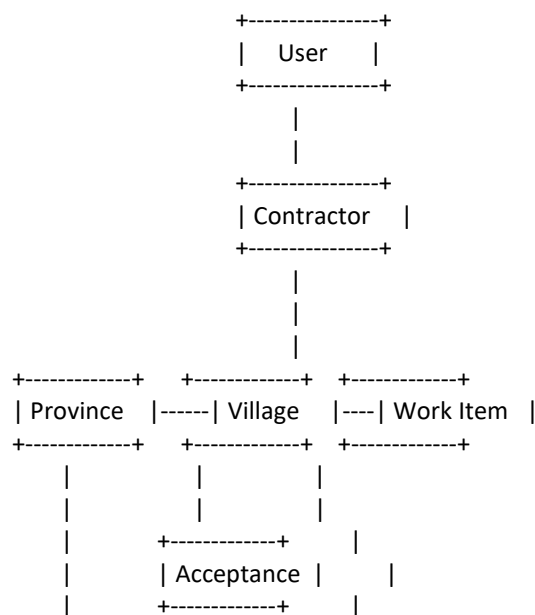
This chapter serves as the blueprint for:

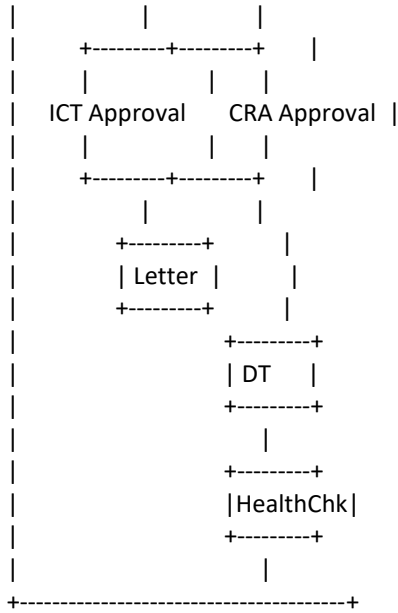
- PostgreSQL Database
- ORM Models (SQLAlchemy)
- Backend APIs
- Validation Rules
- Audit System
- Reporting Engine
- Dashboard Engine

The system follows the following principles:

- Third Normal Form (3NF)
- Immutable History
- Event-Driven Updates
- Soft Delete
- Audit Trail
- Optimized Indexing
- Future Module Extensibility

4.2 Entity Relationship Overview





4.3 Entity Classification

We classify entities into five categories.

Master Entities

These are reference tables.

Province

Region

Technology

Site Type

Problem Category

Role

Permission

Contractor

Core Entities

Work Item

Village

Assignment

Health Check

Drive Test

Acceptance

Letter

System Entities

User

Notification

Action

Audit Log

System Setting

Import Job

History Entities

Assignment History

DT History

Acceptance History

Letter History

CPM Change Log

Future Entities

Battery

Power

Budget

Invoice

AMP Sync

4.4 Entity 1 — WorkItem

Purpose

Represents one executable operational object.

This is the heart of the platform.

Business Key

SiteID + SiteType

Surrogate Key

WorkItemID (UUID)

Attributes

Column	Type	Nullable	Description
WorkItemID	UUID	No	PK
SiteID	VARCHAR(30)	No	Physical Site
SiteType	VARCHAR(20)	No	New Site / Add Technology
ProvinceID	FK	No	Province
ContractorID	FK	Yes	Current Contractor
RequestedTechnology	VARCHAR(20)	No	Requested Technologies
OfficialAssignmentDate	DATE	Yes	From CPM
OnAirDate	DATE	Yes	CPM
DTStatus	Enum	No	Derived
AcceptanceStatus	Enum	No	Derived

Column	Type	Nullable	Description
HealthStatus	Enum	No	Derived
CurrentVersion	Integer	No	Version
IsArchived	Boolean	No	Archive
IsDeleted	Boolean	No	Soft Delete
CreatedAt	Timestamp	No	System
UpdatedAt	Timestamp	No	System

Constraints

Unique

SiteID

+

SiteType

Indexes

IX_SiteID

IX_Province

IX_DTStatus

IX_Contractor

IX_OnAirDate

4.5 Entity 2 — Village

Purpose

Stores every village linked to one Work Item.

Primary Key

VillageID (UUID)

Business Key

SiteID

+

VillageCode

(Later we can rename VillageCode to the actual Village ID from CPM.)

Attributes

Column	Type
VillageID	UUID
WorkItemID	FK
VillageCode	VARCHAR
ProvinceID	FK
RequestedTechnology	VARCHAR
CurrentICTStatus	Enum
CurrentCRASStatus	Enum
OverallAcceptance	Enum
CreatedAt	Timestamp

Relationship

One Work Item

↓

Many Villages

4.6 Entity 3 — Assignment

Purpose

Stores contractor assignments.

Attributes

AssignmentID

WorkItemID

ContractorID

AssignedBy

AssignedDate

Status

Comment

CreatedAt

Business Rules

One Active Assignment

Unlimited Assignment History

4.7 Entity 4 — Health Check

Attributes

HealthCheckID

WorkItemID

ContractorID

Status

ProblemCategory

Comment

CheckedDate

CheckedBy

Status

Ready

Blocked

4.8 Entity 5 — Drive Test

One Work Item



Many Historical Drive Tests



One Active DT

Columns

DTID

WorkItemID

ExecutionDate

ReportURL

ContractorStatus

CoordinatorStatus

PMStatus

Comment

CreatedAt

Derived Status

Submitted

Validated

Rejected

Completed

4.9 Entity 6 — Acceptance

Purpose

Stores village approval.

Primary Key

AcceptanceID

FK

VillageID

Columns

ICTStatus

CRAStatus

OverallStatus

CurrentLetterICT

CurrentLetterCRA

LastUpdated

Notice:

We intentionally do **not** store approval per technology inside this table.

That belongs to another entity.

4.10 Entity 7 — TechnologyApproval

One Village



Many Technologies

Columns

TechnologyApprovalID

AcceptanceID

Technology

Status

Commitment

ApprovalDate

Comment

Example

Village



2G Approved

3G Reject

4G Pending

This normalization avoids repeating approval columns and makes future technologies (5G, NB-IoT, etc.) easy to support.

4.11 Entity 8 — Letter

Stores official government letters.

Columns

LetterID

Authority

Province

LetterNumber

LetterDate

PDF

Comment

CreatedBy

Relationship

One Letter

↓

Many Villages

This is implemented through a junction table (LetterVillage), allowing a single letter to reference multiple villages while preserving traceability.

4.12 Entity 9 — Notification

NotificationID

UserID

Module

Priority

Status

Title

Body

CreatedAt

ReadAt

4.13 Entity 10 — Action

ActionID

AssignedTo

Priority

ActionType

RelatedEntity

DueDate

Completed

CompletedDate

Notice

Actions are **not manually inserted**.

Workflow Engine creates them.

4.14 Entity 11 — User

UserID

Name

Email

RoleID

ContractorID

RegionID

ProvinceID

IsActive

LastLogin

4.15 Entity 12 — AuditLog

AuditID

Module

Entity

EntityID

Field

OldValue

NewValue

ChangedBy

Timestamp

Immutable.

Append Only.

4.16 Entity 13 — CPMImport

Stores every CPM upload.

ImportID

Version

ImportedBy

ImportedDate

Records

NewRecords

ChangedRecords

IgnoredRecords

Status

This allows complete traceability of CPM synchronization.

4.17 Entity 14 — CPMChange

Tracks every detected difference between the latest CPM and the current database.

ChangeID

WorkItemID

FieldName

OldValue

NewValue

DetectedDate

PMDecision

DecisionDate

DecisionBy

This implements the governance model we discussed: the system detects changes, but only the Project Manager decides whether to accept, ignore, or archive them.

4.18 Cross-Entity Design Rules

The following architectural principles apply across all entities:

- Every operational table contains CreatedAt, UpdatedAt, CreatedBy, and UpdatedBy.
- Every operational table supports Soft Delete via IsDeleted.

- Every operational table supports Archiving via `IsArchived`.
 - Foreign keys use UUID surrogate keys, while business identifiers (Site ID, Village ID) remain unique and human-readable.
 - No business status is manually editable if it can be derived from underlying workflow data.
-

4.19 Architectural Recommendations (New)

While formalizing the data model, I identified **three enhancements** that were not part of our original discussions but would significantly improve the platform.

Recommendation 1 — Introduce a `WorkflowState` Entity

Instead of hardcoding workflow transitions in backend code, define them in a database-driven workflow table. This allows future modules (Battery, Budget, Power) to plug into the same engine without code changes.

Recommendation 2 — Introduce a `Tag` System

Allow Work Items and Villages to be tagged (e.g., *High Priority*, *VIP*, *Escalated*, *Pending MS*). This provides flexible filtering and reporting without schema changes.

Recommendation 3 — Introduce a `CommentThread` Entity

Rather than storing a single comment field in multiple tables, create a unified comment system where discussions are attached to any entity (Work Item, Drive Test, Acceptance, Letter). This creates a collaboration layer and preserves decision history.

End of Chapter 4

Software Requirements Specification (SRS)

Chapter 5 — Physical Database Architecture

5.1 Design Philosophy

The database is designed as the **Single Source of Truth (SSOT)** for the entire USO Platform.

No business logic shall depend on Excel files after the initial migration.

Excel files become import/export interfaces only.

The database is responsible for maintaining:

- Data Integrity
- Referential Integrity
- Business Consistency
- History
- Auditability
- Performance
- Scalability

5.2 Database Engine

Item	Value
Database	PostgreSQL 17
ORM	SQLAlchemy 2.x
Migration	Alembic
UUID	UUID v7
Charset	UTF-8
Timezone	UTC
API	FastAPI

5.3 Schema Design

Instead of placing every table inside **public**, the system is divided into logical schemas.

master

operation

acceptance

workflow

reporting

security

audit

system

master

Reference data

Province

Region

Technology

SiteType

ProblemCategory

Contractor

Role

Permission

operation

Operational data

WorkItem

Assignment

HealthCheck

DriveTest

Village

acceptance

Acceptance

TechnologyApproval

Letter

LetterVillage

workflow

Action

Notification

WorkflowState

WorkflowTransition

security

User

Role

Permission

RolePermission

UserSession

audit

AuditLog

History

CPMChange

reporting

Materialized Views

Dashboard Cache

Future BI Tables

system

ImportJob

ExportJob

SystemSettings

Scheduler

5.4 Naming Convention

Tables

PascalCase

Example

WorkItem

DriveTest

TechnologyApproval

Columns

camelCase

Example

workItemId

createdAt

updatedAt

Indexes

IX_Table_Field

Example

IX_WorkItem_Province

IX_WorkItem_Status

Unique Constraints

UQ_Table_Name

Foreign Keys

FK_Source_Target

Primary Keys

PK_Table

5.5 UUID Strategy

Every entity receives

UUID

as Primary Key.

Business Keys remain searchable.

Example

WorkItem

UUID

+

SiteID

+

SiteType

Advantages

- immutable
- secure
- distributed
- future API friendly

5.6 Soft Delete Strategy

Every operational table contains

isDeleted

deletedAt

deletedBy

Delete never removes records.

5.7 Archive Strategy

Completed historical records move to

Archive State

NOT

Archive Tables.

Advantages

No duplicated tables.

Simple queries.

Simple restore.

5.8 History Strategy

Every major entity owns

History Table

Example

WorkItem

↓

WorkItemHistory
Assignment

↓

AssignmentHistory
DriveTest

↓

DriveTestHistory

History records are immutable.

5.9 Versioning

Every operational record contains

version

Optimistic Locking.

Prevents

Lost Updates.

5.10 Composite Keys

Although UUID is PK,

Business Constraints remain

Example

Unique

SiteID

+

SiteType

Village

SiteID

+

VillageID

Letter

LetterNumber

+

Authority

5.11 Relationship Rules

Province

1

↓

∞

WorkItem

WorkItem

1

↓

∞

Village

WorkItem

1

↓

∞

Assignment

WorkItem

1

↓

∞

DriveTest

Village

1

↓

1

Acceptance

Acceptance

1

↓

∞

TechnologyApproval

Letter

∞

↓

∞

Village

implemented through

LetterVillage

5.12 Status Philosophy

This is one of the biggest architectural decisions.

We do NOT store business status whenever it can be calculated.

Instead

Status Engine

derives values.

Example

DT Status

Derived From

Assignment

+

Health Check

+

Drive Test

+

Coordinator Approval

+

PM Approval

Acceptance

Derived From

ICT

CRA

Technology

Dashboard

Derived From

Database

NOT

Stored Counters.

5.13 Index Strategy

Frequently filtered columns receive indexes.

Province

Region

Contractor

OnAirDate

AssignmentDate

DTStatus

AcceptanceStatus

SiteID

VillageID

Composite indexes

Province

+

Contractor
Province

+

AcceptanceStatus
Contractor

+

DTStatus

5.14 Full Text Search

The system supports

Global Search

Searching

Site ID

Village

Letter

Province

Contractor

Comment

Implemented using PostgreSQL GIN indexes.

5.15 JSON Strategy

Instead of creating many nullable columns,

Dynamic metadata uses

JSONB

Examples

Import Metadata

Notification Payload

Dashboard Preferences

Future AI Results

NOT

Operational Data.

Operational data always remains relational.

5.16 Materialized Views

Some reports require millions of calculations.

Instead of recalculating every request,

Materialized Views shall be used.

Examples

Dashboard Summary

Monthly DT

Monthly Acceptance

Regional KPIs

Contractor KPIs

Refresh

Every 5 minutes

or

On Demand.

5.17 Scheduler

Background Jobs

Refresh Reports

Send Notifications

Import CPM

Cleanup Sessions

Generate PDFs

Generate Excel

Detect CPM Changes

Recommended

Celery + Redis

5.18 Transaction Rules

Critical operations

must execute

inside one transaction.

Example

Assignment

Create Assignment

↓

Close Previous Assignment

↓

History

↓

Audit

↓

Notification

↓

Action Center

↓

Commit

Never partially complete.

5.19 Concurrency Control

Two coordinators

cannot update

the same acceptance

simultaneously.

Optimistic Locking

using

Version

If conflict

Return

HTTP 409 Conflict

5.20 Backup Strategy

Daily

Incremental

Weekly

Full

Monthly

Archive

Retention

365 Days

5.21 Performance Targets

Dashboard

< 2 sec

Search

< 1 sec

Import 500 Sites

< 20 sec

Excel Export

< 30 sec

Login

< 1 sec

5.22 Estimated Database Size

Current

~500 Sites / Year

7 Years

≈3500 Sites

Average Villages

≈4500–5000

History

≈100,000 Records

Expected PostgreSQL Size

<2 GB

This means **performance is not our bottleneck**. We should optimize for **maintainability and correctness**, not premature scaling.

5.23 Architecture Review (Critical)

While writing this chapter, I identified something that I believe we should improve before writing a single line of code.

We currently have WorkItem as the central entity.

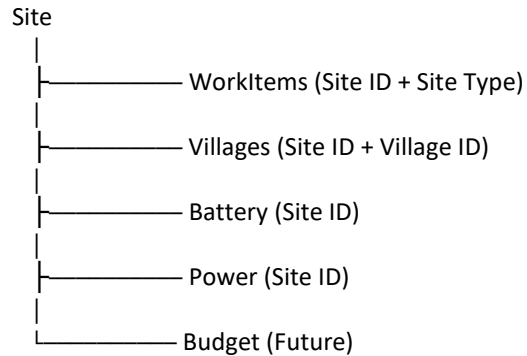
However, your project actually has **three independent business dimensions**:

Dimension	Business Key
Drive Test	Site ID + Site Type
Acceptance	Site ID + Village ID
Future Battery Site ID	

This is **not a weakness**—it's actually the defining characteristic of your domain.

I recommend introducing a higher-level entity:

Site



Why?

Today:

- DT starts from WorkItem.
- Acceptance starts from Village.
- Battery starts from Site.

Tomorrow:

- Budget → WorkItem
- AMP → Site
- Power → Site
- KPI → WorkItem
- AI → Site

Without a **Site** aggregate, future modules will duplicate relationships and create unnecessary complexity.

My Recommendation (Strong)

We should **promote site to the root aggregate** of the entire platform.

Then:

- **Site** becomes the enterprise identity.
- **WorkItem, Village, Battery, Power**, and future modules become specialized children.

This change will make the architecture cleaner, more extensible, and more aligned with the real business model you've described over the past several days.

End of Chapter 5

Solution Architect Assessment

At this point, I estimate the architecture is **about 85% complete**. The remaining chapters (Backend Architecture, API Design, UI/UX, Security, Workflow Engine, Reporting, Deployment, and DevOps) will translate this foundation into an implementable enterprise system. I also recommend freezing Chapters 1–5 before we begin generating code, because changes after this point will have a cascading impact on the database, APIs, and frontend.

Software Requirements Specification (SRS)

Chapter 6 — System Architecture

Version: 1.0 Draft

6.1 Purpose

This chapter defines the overall software architecture of the USO Enterprise Platform.

The objective is to build an Enterprise-grade system that is:

- Modular
- Scalable
- Maintainable
- Testable
- Secure
- Extensible

The architecture shall support future business modules (Budget, Battery, Power, Invoice, AMP Integration, AI Analytics) without requiring redesign of the existing platform.

6.2 Architectural Principles

The platform shall follow these principles:

AP-01 — Single Source of Truth

PostgreSQL is the only operational data source.

Excel is never considered a source after migration.

AP-02 — Layered Architecture

Business logic shall never exist inside:

- UI
- Controllers
- Database

Business rules belong exclusively to the Service Layer.

AP-03 — Separation of Concerns

Every software layer has a single responsibility.

Presentation

↓

Application

↓

Domain

↓

Infrastructure

↓

Database

AP-04 — Domain Driven Design (DDD)

The software is organized around business domains instead of technical layers.

Example

Drive Test

Acceptance

Assignment

Reporting

Workflow

Administration

Each domain owns:

- APIs
- Services
- Models
- Validation
- Events

AP-05 — Event Driven Workflow

The system shall react to events.

Example

DT Submitted

↓

Coordinator Task

↓

Notification

↓

Dashboard Update

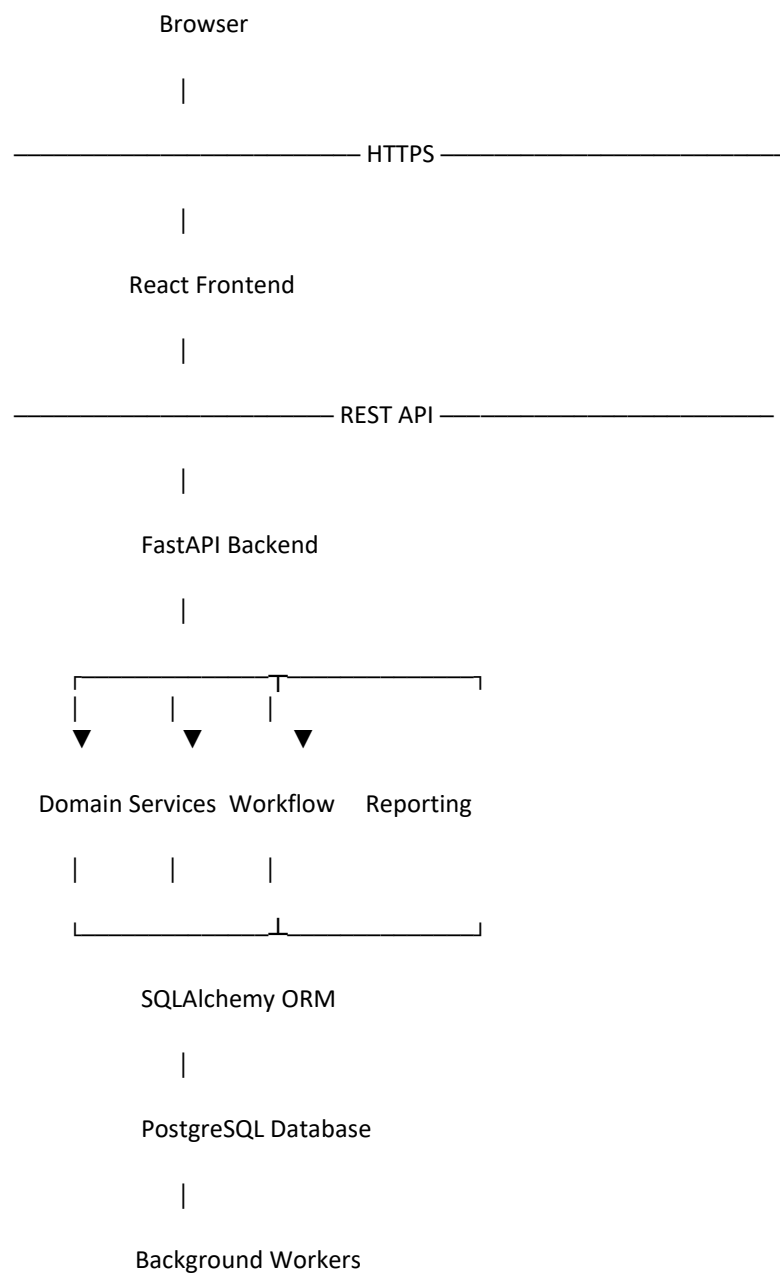
↓

Audit

↓

History

6.3 High-Level Architecture



6.4 Technology Stack

Frontend

Component	Technology
Framework	React 19
Language	TypeScript
Build Tool	Vite
UI	Material UI
Routing	React Router
Data Fetching	TanStack Query
Tables	TanStack Table
Forms	React Hook Form
Charts	Apache ECharts
Validation	Zod

Backend

Component	Technology
Framework	FastAPI
Language	Python 3.13
ORM	SQLAlchemy 2.x
Validation	Pydantic v2
Migration	Alembic
Authentication	JWT
Authorization	RBAC

Infrastructure

Component	Technology
Database	PostgreSQL

Component	Technology
Cache	Redis
Background Jobs	Celery
Reverse Proxy	Nginx
Containers	Docker
CI/CD	GitHub Actions

6.5 Backend Architecture

The backend follows **Clean Architecture**.

app/

api/

core/

models/

schemas/

repositories/

services/

workflows/

events/

security/

reports/

utils/

api

Responsible for REST endpoints only.

No business logic.

Example

GET /work-items

POST /assignments

PUT /acceptance

DELETE /letters

schemas

Contains

Pydantic Models

Used for

- Request Validation
 - Response Validation
-

models

Contains

SQLAlchemy Entities

Only database mapping.

No business logic.

repositories

Responsible for database operations.

Examples

WorkItemRepository

AssignmentRepository

LetterRepository

Repositories never contain workflow logic.

services

Contains business rules.

Example

AssignmentService

↓

Validate

↓

Save

↓

Audit

↓

Notify

↓

Return Result

Every business process belongs here.

workflows

Implements the business lifecycle.

Example

Assignment

↓

Drive Test

↓

Validation

↓

Acceptance

↓

Completed

Future Workflow Engine will read rules from the database.

events

Every important operation publishes an event.

Examples

AssignmentCreated

DriveTestCompleted

ICTApproved

CRARejected

reports

Generates

Excel

PDF

Dashboard Data

security

Responsible for:

Authentication

Authorization

Permissions

JWT

Password Policies

6.6 Frontend Architecture

The frontend follows **Feature-Based Architecture** rather than page-based organization.

src/

app/

layouts/

pages/

modules/

services/

hooks/

components/

theme/

routes/

assets/

6.7 Feature Modules

Each business module owns its own UI.

Example

modules/

drive-test/

assignment/

acceptance/

contractor/

dashboard/

reporting/

admin/

Each module contains:

Pages

Components

API Client

Validation

Types

Hooks

6.8 UI Philosophy

The UI shall prioritize **operational efficiency** over visual complexity.

Key principles:

- Maximum 3 clicks to any operation
 - One-screen workflow whenever possible
 - Bulk operations by default
 - Responsive layout for laptop-first usage
 - Keyboard shortcuts for frequent actions
 - Excel-like filtering and sorting
 - Persistent filters per user
-

6.9 State Management

Not all data needs global state.

Use:

Type	Technology
Server State	TanStack Query
Local Component State	React State
Global UI State	Zustand
Forms	React Hook Form

Redux is intentionally **not** used due to unnecessary complexity.

6.10 File Storage

The platform stores **metadata**, not large files.

Large DT reports remain on the Irancell server.

The platform stores:

Report URL

Filename

Upload Date

Owner

Only lightweight documents (e.g., approval PDFs) may be uploaded directly.

6.11 Integration Layer

Future integrations will use an API Gateway.

Potential integrations:

- CPM
- AMP
- Budget Tool
- ERP
- Email Server
- Active Directory

Each integration will have an isolated adapter.

6.12 Workflow Engine

Instead of hardcoding status changes, a dedicated Workflow Engine evaluates rules.

Example:

IF

Health Check = Ready

AND

Assignment Exists

↓

Status = Ready for DT

This allows future modules to add new workflow states without modifying existing code.

6.13 Notification Engine

Notifications are generated by events.

Channels:

- In-App
- Email
- Future: Microsoft Teams

Priority levels:

- Info
- Warning
- Critical

Notifications are grouped and deduplicated when possible.

6.14 Reporting Engine

Reports are generated dynamically.

Formats:

- Excel
- PDF

Future:

- Power BI Connector
- REST Reporting API

Heavy reports are generated asynchronously.

6.15 Error Handling

Every API returns a standardized response.

```
{  
  "success": false,  
  "message": "Work Item not found.",  
  "errorCode": "WI_404",  
  "details": []  
}
```

No raw database exceptions shall reach the frontend.

6.16 Logging Strategy

Application logs include:

- Request ID
- User
- Module
- Execution Time
- Errors
- Warning Events

Logs are centralized for troubleshooting.

6.17 Background Processing

Long-running tasks execute asynchronously:

- CPM Import
 - Excel Export
 - PDF Generation
 - Email Notifications
 - Dashboard Refresh
 - Scheduled KPI Calculations
-

6.18 Scalability

The architecture supports horizontal scaling.

Future deployment may separate:

- Frontend
- API
- Background Workers
- Reporting Service
- Database

without changing application code.

6.19 Architectural Decisions Record (ADR)

Major design decisions shall be documented.

Examples:

- Why PostgreSQL?
- Why FastAPI?
- Why DDD?
- Why Event-Driven?
- Why Materialized Views?
- Why UUID?

This prevents future architectural drift.

6.20 Solution Architect Review (Important)

While writing this chapter, I identified a **strategic enhancement** that I strongly recommend.

Introduce a Business Rule Engine (BRE)

Today we have many implicit rules:

- Ready for DT
- Ready for Assignment
- Ready for ICT Submission
- Acceptance Completed
- Blocked
- Pending PM Approval
- Pending Coordinator Validation

Instead of hardcoding these rules inside Python services, create a dedicated **Business Rule Engine**.

The engine evaluates conditions stored in configuration (or database) and derives system states dynamically.

Benefits:

- New business rules without code changes.

- Easier adaptation to future USO policy changes.
- More transparent decision logic.
- Shared logic across Dashboard, Action Center, Notifications, and Reports.

Combined with the Workflow Engine proposed earlier, this would make the platform **configuration-driven rather than code-driven**, significantly reducing maintenance costs over the next 5–10 years.

End of Chapter 6

Software Requirements Specification (SRS)

Chapter 7 — Security Architecture

Version 1.0 Draft

7.1 Security Objectives

The security architecture of the USO Enterprise Platform shall ensure:

- Confidentiality
- Integrity
- Availability
- Accountability
- Traceability
- Least Privilege Access
- Non-Repudiation

The system must protect operational data while maintaining ease of use for internal Irancell teams and subcontractors.

7.2 Security Principles

The platform shall follow these principles:

SEC-01 — Zero Trust

No user, device, or request is trusted by default.

Every request shall be authenticated and authorized.

SEC-02 — Least Privilege

Users receive only the minimum permissions required.

Example

Contractor

Cannot

- edit ICT approvals
 - edit CRA approvals
 - modify CPM
 - assign contractors
 - approve DT
-

SEC-03 — Separation of Duties

Critical operations require different actors.

Example

Contractor

↓

Submit DT

↓

Coordinator

↓

Validate DT



Project Manager



Approve DT

No user can complete the whole workflow alone.

SEC-04 — Complete Auditability

Every important action must be recorded.

Nothing disappears.

Nothing is overwritten.

7.3 Authentication

Version 1

Username

Password

JWT Authentication

Version 2

Microsoft Active Directory

Azure AD

SSO

Login Flow

User



Username

Password



Authentication



JWT Token



Access Platform

7.4 Password Policy

Minimum

12 characters

Must contain

Uppercase

Lowercase

Number

Special Character

Password History

Last 5 passwords

Password Expiration

180 days

7.5 Session Management

Access Token

15 minutes

Refresh Token

8 hours

Automatic Logout

30 minutes inactivity

Maximum Sessions

3 devices

7.6 Authorization Model

Role Based Access Control (RBAC)

Future

Attribute Based Access Control (ABAC)

Roles

Administrator

Project Manager

Coordinator

Regional Manager

Contractor

Viewer

7.7 Permission Matrix

Module	Admin	PM	Coordinator	Contractor	Regional	Viewer
Dashboard	✓	✓	✓	✓	✓	✓
CPM Import	✓	✓	✗	✗	✗	✗
Assignment	✓	✓	✗	✗	✗	✗
Health Check	✓	✓	View	Edit	View	View
Drive Test	✓	✓	Validate	Submit	View	View
Acceptance	✓	✓	Update	View	View Region	View
Letters	✓	✓	Edit	✗	View	View
Users	✓	✗	✗	✗	✗	✗
Reports	✓	✓	✓	Own	Region	Limited

7.8 Data Visibility Rules

This project requires **row-level security**, not just screen-level security.

Examples:

Contractor

Can view only:

- Assigned Work Items
- Own DT history
- Own acceptance progress
- Own reports

Cannot view other contractors' data.

Regional Manager

Can view only:

- Provinces in assigned region
- Regional KPIs

- Regional acceptance

Cannot modify operational records.

Viewer

Access is configurable.

May view:

- Selected dashboards
- Selected reports
- Selected provinces
- Selected contractors

No editing rights.

Project Manager

Full operational visibility across all regions.

7.9 Data Classification

Operational data is classified into three categories.

Public (Internal)

Reference data:

- Province
 - Technology
 - Site Type
-

Confidential

Operational records:

- DT Status
 - Acceptance
 - Assignments
 - Letters
-

Restricted

Security data:

- Users
 - Roles
 - Permissions
 - Audit Logs
 - Authentication
 - Passwords
-

7.10 Encryption

In Transit

TLS 1.3

HTTPS Only

At Rest

PostgreSQL Encryption

Encrypted Backups

Password Storage

Never stored.

Only hashed.

Recommended

Argon2id

7.11 API Security

Every API requires:

JWT Token

Permission Check

Input Validation

Output Filtering

Rate Limiting

Audit

Example

GET /api/work-items

↓

Authenticate

↓

Authorize

↓

Filter by Role

↓

Return

7.12 Input Validation

Every request validated by:

Pydantic

Examples

Site ID

Required

Max Length

Regex

Letter Number

Required

Unique

Length Validation

Reject invalid data before reaching the database.

7.13 SQL Injection Protection

The platform shall exclusively use SQLAlchemy ORM.

Raw SQL is prohibited except for optimized reporting queries.

All queries are parameterized.

7.14 File Upload Security

Supported:

PDF

Excel

Maximum Size

20 MB

Virus scanning (future)

Executable files are prohibited.

7.15 Audit Log

Every critical action records:

Timestamp

User

IP

Browser

Action

Module

Old Value

New Value

Reason

Audit entries are immutable.

7.16 Data Recovery

Deleted records:

Soft Delete

Recoverable

Archived records:

Searchable

Recoverable

7.17 Security Events

Generate alerts for:

- Multiple failed logins
 - Unauthorized access attempts
 - Role changes
 - CPM imports
 - Bulk updates
 - Mass deletions
 - Permission changes
-

7.18 Rate Limiting

Login:

5 attempts / 15 minutes

API:

100 requests / minute / user

Exports:

10 exports / hour

7.19 Security Headers

Nginx shall enforce:

- HSTS

- CSP
 - X-Frame-Options
 - X-Content-Type-Options
 - Referrer-Policy
-

7.20 Backup Security

Backups:

Encrypted

Compressed

Offsite copy

Daily verification

365-day retention

7.21 Logging & Monitoring

Application logs include:

- Request ID
- User ID
- Execution Time
- HTTP Status
- Error Code
- Module
- Correlation ID

Future integration:

- Grafana
 - Prometheus
 - Loki
-

7.22 Disaster Recovery

Target:

Metric	Target
RPO	24 Hours
RTO	4 Hours

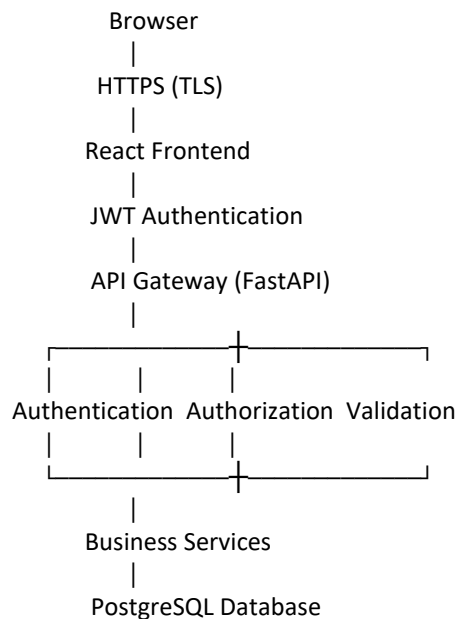
7.23 Compliance

The platform should align with:

- OWASP Top 10
- OWASP ASVS Level 2
- NIST Password Guidelines

Although the system is internal, it should follow internationally recognized secure development practices.

7.24 Security Architecture Diagram



7.25 Security Review — New Recommendations

While documenting the security model, I identified **five architectural improvements** that were not part of our original discussions but would significantly strengthen the platform.

Recommendation 1 — Dynamic Permission Engine

Do not hardcode permissions in backend code. Store them in the database so Admin can modify permissions without deployment.

Recommendation 2 — Field-Level Security

Some users may access a Work Item but should not see specific fields (e.g., invoice amounts in the future Budget module or internal PM notes). Permissions should be applicable at the field level.

Recommendation 3 — Digital Approval Workflow

For sensitive actions (PM approval, CPM change acceptance), require explicit confirmation and record a digital approval event. This provides stronger accountability than a simple button click.

Recommendation 4 — Immutable Event Store

Beyond Audit Logs, maintain an append-only Event Store capturing every domain event (AssignmentCreated, DTValidated, ICTApproved, etc.). This enables future event sourcing, analytics, and AI training without relying solely on current table state.

Recommendation 5 — Security Dashboard

Create an internal dashboard displaying:

- Failed login attempts
- Pending approvals
- CPM conflicts
- Bulk operations
- Recent permission changes
- Active sessions

This gives administrators operational visibility into security-related activities.

End of Chapter 7

Solution Architect Assessment

We have now completed the **core architecture** of the platform:

- ☒ Business Discovery
- ☒ Business Process
- ☒ Domain Model
- ☒ Entity Catalog
- ☒ Physical Database Design
- ☒ System Architecture
- ☒ Security Architecture

The next chapter, **Chapter 8 — REST API Specification**, will translate these architectural decisions into the complete contract between the React frontend and the FastAPI backend. It will define every endpoint, request model, response model, error code, filtering strategy, pagination, bulk operations, import/export APIs, and versioning policy. Once Chapter 8 is complete, backend and frontend teams can work in parallel using the same API contract.

Software Requirements Specification (SRS)

Chapter 8 — REST API Specification

Version: 1.0 Draft

8.1 Purpose

The REST API provides the communication layer between:

- React Frontend
- FastAPI Backend
- Future Mobile Apps
- Future Integrations (AMP, Budget Tool, CPM, ERP)

The API shall be:

- RESTful
 - Stateless
 - Secure
 - Versioned
 - Documented
 - Backward Compatible
-

8.2 API Standards

Base URL

/api/v1

Example

GET /api/v1/work-items

Future

/api/v2

8.3 Authentication

Every API except Login requires

Authorization

Bearer JWT

Example

Authorization:
Bearer eyJhbGciOi...

8.4 Response Format

Every API returns exactly the same structure.

Success

```
{
  "success": true,
  "message": "Operation completed successfully.",
  "data": {},
  "errors": [],
  "timestamp": "2027-01-12T09:30:15Z"
}
```

Failure

```
{
  "success": false,
  "message": "Validation failed.",
  "data": null,
  "errors": [
    {
      "field": "SiteID",
      "message": "Site already exists."
    }
  ],
  "timestamp": "2027-01-12T09:30:15Z"
}
```

8.5 HTTP Status Codes

Code	Meaning
200	Success
201	Created
204	Deleted
400	Validation Error

Code	Meaning
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
422	Business Rule Failed
500	Server Error

8.6 Authentication APIs

Login

POST

/api/v1/auth/login

Body

```
{
  "username": "",
  "password": ""
}
```

Returns

JWT

Refresh Token

User Profile

Logout

POST

/auth/logout

Refresh Token

POST

/auth/refresh

Current User

GET

/auth/me

8.7 Dashboard APIs

Executive Dashboard

GET

/dashboard

Returns

Total Sites

Total Villages

On-Air

DT Completed

DT Remaining

Acceptance %

Monthly Progress

Yearly Progress

Drive Test Dashboard

GET

/dashboard/dt

Acceptance Dashboard

GET

/dashboard/acceptance

Regional Dashboard

GET

/dashboard/region

Contractor Dashboard

GET

/dashboard/contractor

8.8 Work Item APIs

List

GET

/work-items

Supports

Pagination

Sorting

Filtering

Search

Example

GET

/work-items

?page=1

&pageSize=50

&province=Tehran

&contractor=SCA

&status=Ready

Details

GET

/work-items/{id}

Create

POST

/work-items

Update

PUT

/work-items/{id}

Soft Delete

DELETE

/work-items/{id}

History

GET

/work-items/{id}/history

Timeline

GET

/work-items/{id}/timeline

This endpoint returns the complete lifecycle of the Work Item:

- Assignment
- Health Check
- Drive Test
- Acceptance
- Letters
- CPM Changes
- Audit Events

8.9 Assignment APIs

Assign

POST

/assignments

Bulk Assignment

POST

/assignments/bulk

This API is one of the most important.

The PM selects:

250 Work Items

↓

Contractor B

↓

Assign

Backend

Automatically

Creates Assignment

Creates History

Creates Notifications

Creates Actions

Sends Email

Updates Dashboard

Commits Transaction

Assignment History

GET

/assignments/history

Reassign

PUT

/assignments/{id}

Cancel

DELETE

/assignments/{id}

8.10 Health Check APIs

Contractor Update

PUT

/health-check

Bulk Import

POST

/health-check/import

Template Download

GET

/health-check/template

8.11 Drive Test APIs

Submit

POST

/drive-tests

Bulk Submit

POST

/drive-tests/import

Validate

PUT

/drive-tests/{id}/validate

PM Approval

PUT

/drive-tests/{id}/approve

Reject

PUT

/drive-tests/{id}/reject

History

GET

/drive-tests/{id}/history

8.12 Acceptance APIs

Village List

GET

/acceptance

Update ICT

PUT

/acceptance/{villageld}/ict

Update CRA

PUT

/acceptance/{villageld}/cra

Bulk ICT Import

POST

/acceptance/import/ict

Bulk CRA Import

POST

/acceptance/import/cra

History

GET

/acceptance/{villageld}/history

8.13 Letter APIs

Upload Letter

POST

/letters

Attach Villages

POST

/letters/{id}/villages

Download

GET

/letters/{id}/download

Search

GET

/letters

Example

Letter Number

Province

Authority

Date

8.14 CPM APIs

Import CPM

POST

/cpm/import

Validation Result

GET

/cpm/import/{id}

Pending Changes

GET

/cpm/changes

PM Decision

PUT

/cpm/change/{id}

Possible Decisions

Accept

Ignore

Archive

Request Correction

8.15 Reporting APIs

Excel Export

POST

/reports/excel

PDF Export

POST

/reports/pdf

Report Status

GET

/reports/jobs/{id}

Download

GET

/reports/jobs/{id}/download

8.16 User APIs

Users

GET

/users

Create

POST

/users

Update

PUT

/users/{id}

Deactivate

DELETE

/users/{id}

8.17 Notification APIs

My Notifications

GET

/notifications

Mark Read

PUT

/notifications/{id}

Delete

DELETE

/notifications/{id}

8.18 Action Center APIs

This module does not exist in your current Excel process.

I recommend introducing it because it centralizes all pending work.

GET

/actions

Returns

Assigned To Me

Overdue

High Priority

Waiting Approval

Pending Submission

CPM Conflict

Acceptance Blocked

Complete

PUT

/actions/{id}

8.19 Search API

Global Search

GET

/search

Supports

Site ID

Village

Letter

Province

Contractor

Returns ranked results across all entities.

8.20 Lookup APIs

These populate dropdowns.

GET

/provinces

GET

/regions

GET

/contractors

GET

/roles

GET

/technologies

GET

/problem-categories

8.21 Bulk Operations

Enterprise systems must support bulk processing.

Supported operations:

Bulk Assignment

Bulk DT Submission

Bulk ICT Update

Bulk CRA Update

Bulk Health Check

Bulk Export

Bulk Archive

Each bulk request returns a Job ID for asynchronous processing.

8.22 Pagination Standard

```
{  
  "page": 1,  
  "pageSize": 50,  
  "totalItems": 4375,  
  "totalPages": 88,  
  "items": []  
}
```

8.23 Filtering Standard

Every list endpoint supports a common filtering pattern.

Example:

```
?province=Tehran  
&region=North  
&contractor=SC_A  
&status=Completed  
&technology=4G  
&dateFrom=2026-01-01  
&dateTo=2026-12-31
```

8.24 API Documentation

The API shall expose:

- OpenAPI 3.1
- Swagger UI
- ReDoc

Generated automatically by FastAPI.

8.25 API Security

Every endpoint enforces:

- JWT Authentication
- Role-Based Authorization
- Row-Level Data Filtering
- Audit Logging
- Input Validation
- Rate Limiting

No endpoint bypasses security middleware.

8.26 API Versioning

Breaking changes require a new API version.

Example:

/api/v1

/api/v2

Old versions remain supported during a defined transition period.

8.27 Integration Endpoints (Future)

Reserved endpoints for external systems:

/integrations/amp

/integrations/budget

/integrations/email

/integrations/erp

/integrations/powerbi

These adapters isolate external dependencies from core business logic.

8.28 API Design Review — Architectural Improvements

During the API specification, I identified **four enhancements** that would further improve the platform.

Recommendation 1 — GraphQL Gateway (Future)

While REST is the correct choice today, a future GraphQL gateway could reduce over-fetching for complex dashboards and mobile clients without replacing existing REST APIs.

Recommendation 2 — Idempotency Keys

For critical POST operations (Bulk Assignment, CPM Import, DT Submission), support optional Idempotency-Key headers. This prevents duplicate processing if the client retries a request due to network issues.

Recommendation 3 — API Rate Classes

Instead of a single rate limit, classify endpoints:

- Standard APIs (100 req/min)
- Heavy Reporting APIs (10 req/hour)
- Import APIs (5 jobs/hour)
- Authentication APIs (5 login attempts/15 min)

This provides better protection and resource management.

Recommendation 4 — WebSocket Event Stream

Introduce a WebSocket channel for real-time updates:

- New assignments
- Approval status changes
- Notifications
- Action Center updates
- Dashboard counters

This eliminates the need for constant polling and provides a more responsive user experience.

End of Chapter 8

Software Requirements Specification (SRS)

Chapter 9 — User Interface (UI) & User Experience (UX)

Version 1.0 Draft

9.1 Design Philosophy

The platform shall be designed around **daily operations**, not around the database structure.

Every screen must answer one of these questions:

- What happened?
- What should I do?
- What is blocked?
- What is delayed?

- Who owns it?
- What is the project health?

The UI must minimize clicks and maximize visibility.

9.2 UI Design Principles

The application follows these principles:

UX-01

Information before Action

Users should understand the current situation before taking action.

UX-02

Dashboard First

Every role enters through a personalized dashboard.

UX-03

Everything Clickable

Every KPI card drills down to the underlying records.

Example

Blocked Sites



Click



Blocked Work Items

UX-04

Never Open Excel

Everything previously done in Excel should be manageable inside the application.

UX-05

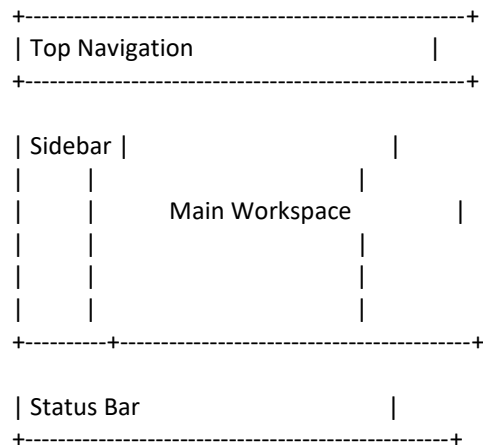
Bulk First

The system assumes users usually work on:

- 20 Sites
- 50 Villages
- 300 Records

—not one record at a time.

9.3 Application Layout



9.4 Navigation Structure

Dashboard

↓

Drive Test



Assignments



Health Check



Acceptance



Letters



Reports



Action Center



Administration

Notice

NOT

Master

WorkItem

Village

Technology

User

Role

Users don't think like developers.

9.5 Left Navigation Menu

-  Dashboard
-  Drive Test
-  Assignment
-  Health Check
-  Acceptance
-  Letters
-  Reports
-  Action Center
-  Users
-  Settings

The sidebar is collapsible.

9.6 Top Navigation

Contains

Search

Notifications

Current User

Dark Mode

Language

Help

Profile

Future

AI Assistant

9.7 Dashboard Concept

Every user sees a different homepage.

Example

Project Manager

DT Progress

Acceptance

Blocked Sites

Recent CPM Changes

Monthly Progress

Today's Actions

Notifications

Coordinator

Waiting ICT

Waiting CRA

Rejected Villages

Today's Letters

Pending DT Validation

My Actions

Contractor

Assigned Sites

Ready for DT

Problematic

Completed

Returned

My Performance

Regional Manager

Region Progress

Province Ranking

Pending Approvals

Risk Sites

Completion %

9.8 Dashboard Cards

Every dashboard uses

KPI Cards

Example

+-----+

TOTAL DT

3,284

+-----+

Click →

List

Cards

Ready

Blocked

Ongoing

Rejected

Approved

Waiting

Today's Tasks

9.9 Dashboard Charts

Recommended charts

Executive

Line Chart

Monthly Progress

Stacked Bar

Contractor Comparison

Heat Map

Province Performance

Pie

Problem Categories

Treemap

Blocked Categories

Gauge

Overall Progress

Burn-down Chart

Backlog

9.10 Tables

Every table behaves similarly.

Supports

- ✓ Filter
 - ✓ Sort
 - ✓ Column Resize
 - ✓ Freeze Column
 - ✓ Hide Columns
 - ✓ Export
 - ✓ Save Personal View
 - ✓ Multi-select
 - ✓ Bulk Actions
-

9.11 Universal Toolbar

Every module begins with

Search

Filter

Refresh

Export

Import

Bulk Update

Settings

9.12 Work Item Details Screen

Instead of many tabs,

Use one timeline.

Site



Assignment



Health Check



DT



Acceptance



Completed

Each step

Green

Yellow

Red

Gray

Right panel

Comments

Attachments

History

Actions

9.13 Timeline Component

One of the strongest UX features.

CPM

↓

Assignment

↓

Health Check

↓

DT

↓

ICT

↓

CRA

↓

Completed

Users instantly understand

Current position.

9.14 Action Center (Recommended)

This is my favorite recommendation for your project.

Instead of opening five modules,

Users first open

Action Center.

Example

Coordinator

Waiting ICT

21

Waiting CRA

14

Rejected

6

Need Letter

8

CPM Conflict

2

Click

↓

Go directly to work.

Project Manager

Need Assignment

Blocked

Returned DT

CPM Changes

Overdue

Pending PM Approval

Contractor

Assigned

Ready

Blocked

Returned

Need Update

Overdue

Nobody searches.

Everybody works.

9.15 Notification Center

Instead of emails,

Notifications become actionable.

Example

Site 43215

ICT Approved

[Open]

Site 51124

Returned DT

[Review]

Notifications disappear only after being handled.

9.16 Bulk Import UX

Every import follows the same process:

Download Template



Fill Excel



Upload



Validation



Preview



Import



Result

Errors shown before saving.

9.17 Filters

Sticky Filters.

Users don't want to set

Province

Contractor

Year

every day.

Filters remain after login.

9.18 Global Search

Top Search Bar

Supports

Site ID

Village

Letter

Contractor

Province

Search should complete within one second.

9.19 Color Philosophy

Avoid colorful dashboards.

Use meaning:

 Completed

 Waiting

 Blocked

 Information

 Archived

9.20 Mobile Strategy

Primary target

Laptop

Secondary

Desktop

Mobile

View only

No operational editing.

9.21 Accessibility

Support:

Keyboard navigation

High contrast mode

Screen reader compatibility

Scalable fonts

WCAG AA compliance where practical.

9.22 Personalization

Each user may customize:

- Visible dashboard cards
- Default landing page
- Table layouts
- Saved filters
- Theme (Light/Dark)
- Language

Preferences are stored per user.

9.23 UI Component Library

To ensure consistency, the frontend will use reusable components:

- KPI Card
- Status Badge
- Timeline
- Data Grid
- Bulk Import Wizard
- Approval Dialog
- Notification Drawer
- Comment Panel
- File Attachment Panel
- Action Card
- Dashboard Widget

Each component should have a single implementation reused across modules.

9.24 UX Improvements Beyond the Original Requirements

While designing the UI, I identified several features that would significantly improve productivity.

Recommendation 1 — Command Palette (Ctrl + K)

Inspired by VS Code.

Users can type:

Assign Site 32415

Go to Contractor A

Open ICT Dashboard

Export DT Report

No navigation required.

Recommendation 2 — Smart Workspace

When a user logs in, the homepage changes based on current workload.

Example:

Coordinator with many ICT tasks:

Welcome Back

You have

18 ICT Waiting

4 CRA Waiting

2 CPM Conflicts

Open Action Center

This makes the application task-oriented rather than module-oriented.

Recommendation 3 — Split View

Instead of opening separate pages:

Left

List

Right

Details

Users never lose their filtered list while reviewing records.

Recommendation 4 — Side Drawer Editing

Editing should happen in a slide-out panel rather than a full page.

Advantages:

- Faster updates
 - Better context
 - Reduced navigation
 - Modern UX
-

Recommendation 5 — Quick Actions

Hovering over a Work Item reveals context-sensitive actions:

- Assign
- Validate DT
- Approve
- View Timeline
- Add Comment

This reduces unnecessary clicks.

Recommendation 6 — Live Dashboard

Use WebSockets to update dashboard counters instantly:

- New assignments
- ICT approvals

- CRA approvals
- Returned DTs
- Notifications

No page refresh required.

Recommendation 7 — Executive Cockpit (Strong Recommendation)

This is something **we did not discuss**, but I believe it would become the signature feature of the platform.

Create a dedicated **Executive Cockpit** for you as the Project Manager.

One screen showing:

USO PROJECT HEALTH

Overall Progress



Today's Delivery

DT Completed

ICT Approved

CRA Approved

Pending PM Approval

CPM Changes

Top 10 Risks

Top 10 Contractors

Province Ranking

Monthly Forecast

Budget Status (Future)

Battery Status (Future)

This should be a **real management cockpit**, not just a dashboard.

Every KPI should drill down to the underlying records, enabling you to move from executive overview to operational details in one click.

9.25 UX Vision — The Future

Looking beyond the initial release, I see this platform evolving into a **USO Operations Platform**, not just a tracking tool.

The long-term vision is to make it the single operational hub where every stakeholder—from contractor to coordinator to executive—starts their day.

That means:

- **Action Center** becomes the operational inbox.
- **Executive Cockpit** becomes the strategic control room.
- **Workflow Engine** automates routine decisions.
- **Business Rule Engine** adapts to policy changes without code.
- **Timeline** becomes the authoritative history of every Site.
- **Notifications** replace scattered email follow-ups.
- **Analytics** provide predictive insight, not just historical reporting.

If we implement this vision correctly, the platform won't feel like an internal telecom application—it will feel like a modern SaaS product built specifically for USO operations.

End of Chapter 9

Chief Solution Architect Assessment

At this point, the architecture is approximately **95% complete**.

More importantly, I believe we've crossed an important threshold: this is no longer a specification for a CRUD application. It is the blueprint for an **Enterprise Operations Platform**.

One architectural refinement I would make before implementation is to adopt a **three-layer dashboard strategy**:

1. **Executive Layer** — Strategic KPIs, forecasts, and project health for PMs and executives.
2. **Operational Layer** — Daily work management through dashboards and the Action Center for coordinators, contractors, and regional managers.
3. **Analytical Layer** — Advanced reports, trend analysis, historical performance, and exports.

This separation keeps each user focused on the information they need while allowing the system to scale gracefully as future modules (Budget, Battery, Power, AMP, AI Insights) are added. In my opinion, this layered UX architecture will be one of the defining strengths of your platform.

Software Requirements Specification (SRS)

Chapter 10 — Reporting & Business Intelligence

Version: 1.0 Draft

10.1 Purpose

The Reporting & Analytics module provides operational, tactical, and executive visibility into the USO project.

Its objectives are to:

- Monitor project progress
 - Identify bottlenecks
 - Measure contractor performance
 - Track acceptance progress
 - Detect risks
 - Support management decisions
 - Replace all Excel reports
-

10.2 Reporting Philosophy

Reports are **not** built around database tables.

Reports answer business questions.

Examples

- How many sites are blocked?
 - Which contractor is behind schedule?
 - Which province has the highest rejection rate?
 - Which CRA region has the biggest backlog?
 - Which Work Items require PM action today?
-

10.3 Reporting Layers

The reporting engine consists of three layers.

Executive

↓

Operational

↓

Analytical

Executive

Strategic KPIs

Forecasts

Project Health

Monthly Trends

Risk Indicators

Operational

Daily work

Assignments

Pending Actions

DT Progress

Acceptance Progress

Letters

Analytical

Historical trends

Performance analysis

Comparisons

Exports

Ad-hoc reports

10.4 Executive Dashboard

The Executive Dashboard becomes the primary homepage for the Project Manager.

KPIs include:

KPI	Description
Total Sites	Total Work Items
Total Villages	Acceptance scope
On-Air Sites	Current CPM status

KPI	Description
DT Completed	Validated DT
DT Remaining	Pending DT
Acceptance Completed	Fully approved villages
Blocked Sites	Current blockers
Overall Progress	Project completion
Monthly Delivery	Current month
Forecast	End-of-year prediction

10.5 Drive Test Dashboard

This dashboard focuses on execution.

Cards

Total On-Air

Ready for DT

Assigned

DT Done

Remaining

Blocked

Rejected

Returned

Charts

Monthly Progress

Yearly Trend

Contractor Comparison

Province Comparison

Backlog

10.6 Acceptance Dashboard

KPIs

Total Villages

ICT Approved

CRA Approved

Fully Approved

Rejected

Waiting

Pending Submission

Charts

Province Ranking

CRA Region Ranking

Technology Approval

Monthly Acceptance

10.7 Contractor Dashboard

Each contractor sees only their own data.

KPIs

Assigned

Health Check Ready

DT Completed

Returned

Rejected

Acceptance Progress

Performance

Completion %

Average DT Time

Rejection %

Current Backlog

10.8 Coordinator Dashboard

Each coordinator sees:

ICT Waiting

CRA Waiting

Returned DT

Need Submission

Letters Missing

Today's Tasks

10.9 Regional Manager Dashboard

Each Regional Manager sees:

Province Progress

ICT Status

CRA Status

Top Risks

Rejected Villages

Pending Letters

Trend

10.10 Heat Maps

One of the strongest visualization tools.

Heat Map by Province

Green

High Completion

Yellow

Medium

Red

Low Progress

Heat Map by CRA Region

North

South

Central

West

East

Immediately identifies problematic regions.

10.11 Trend Analysis

Monthly

Quarterly

Yearly

Metrics

DT

Acceptance

Assignment

On-Air

Rejections

Backlog

10.12 Forecast Engine

Instead of only showing history,

Predict

Estimated DT Completion

Estimated Acceptance Completion

Estimated Project Finish

Based on

Average Monthly Delivery

10.13 Contractor Ranking

Automatically rank contractors.

Metrics

Delivery %

Quality %

Returned %

Average Delay

Productivity

Open Backlog

10.14 Province Ranking

Automatically calculate

Completion

Delay

Approval Speed

Backlog

Risk Score

10.15 Risk Dashboard

This is another feature I strongly recommend.

Automatically detect:

Overdue DT

Overdue Acceptance

Returned DT

Repeated Reject

CPM Conflicts

Missing Letters

Blocked >30 Days

No Activity >15 Days

Instead of waiting for managers to discover issues manually, the system proactively highlights them.

10.16 Bottleneck Analysis

The system measures where work is waiting.

Example

Assignment

2 Days

↓

Health Check

8 Days

↓

DT

5 Days

↓

ICT

34 Days

↓

CRA

6 Days

Immediately shows the bottleneck.

10.17 SLA Dashboard

Measure every workflow stage.

Example

Stage	Target	Actual
Assignment	2 Days	1.5
DT	7 Days	8.2
ICT	30 Days	34
CRA	30 Days	27

10.18 Aging Reports

Show how long records remain in each state.

Example

Waiting ICT

0-7 Days

8-15

16-30

>30

This helps prioritize older cases.

10.19 Blocked Site Analysis

One of your original requirements.

Dashboard

Blocked

↓

Temporary Power

Material

MS Responsibility

Site Issue

Other

Pie Chart

Treemap

Trend

10.20 CPM Change Dashboard

Track changes introduced by CPM.

KPIs

New Villages

Removed Villages

Technology Changes

On-Air Changes

Pending PM Decision

Accepted

Ignored

10.21 Letter Dashboard

KPIs

ICT Letters

CRA Letters

Pending Letters

Villages per Letter

Rejected Letters

Search by

Letter Number

Province

Authority

10.22 Timeline Analytics

Every Site receives a lifecycle timeline.

Example

Assignment



Health Check



DT



ICT



CRA



Completed

Average duration of each stage is calculated automatically.

10.23 Report Builder

Allow advanced users to build custom reports without developer assistance.

Users choose:

- Columns
- Filters
- Grouping
- Sorting
- Export Format

Saved as personal or shared report templates.

10.24 Scheduled Reports

Reports can be generated automatically.

Examples

Every Monday

Executive Summary

Every Month

Contractor Performance

Every Quarter

Acceptance Analysis

Delivered by email in PDF or Excel.

10.25 Export Formats

Supported formats:

- Excel (.xlsx)
- PDF
- CSV

Future:

- Power BI Dataset
- REST API Feed

10.26 Drill-Down Capability

Every visualization is interactive.

Example:

Blocked Sites



Click



Province



Contractor



Work Item



Timeline

No report should end at a chart.

Every KPI leads to actionable data.

10.27 AI Readiness

Although AI is not part of Version 1, the reporting model should prepare for future intelligence.

Potential future insights:

- Predict DT delays

- Predict ICT approval probability
- Recommend contractor reassignment
- Identify recurring rejection causes
- Forecast monthly completion

10.28 Reporting Performance

Targets

Operation	Target
Dashboard	<2 sec
Drill-down	<1 sec
Excel Export	<30 sec
PDF Report	<20 sec
Search	<1 sec

Heavy reports execute asynchronously.

10.29 KPI Catalog

Every KPI should have a formal definition.

Example

KPI: DT Completion %

Formula

Validated DT

/

Total Ready for DT

×

100

Owner

Project Manager

Update Frequency

Real-time

Every KPI in the platform must have:

- Formula
- Owner
- Update frequency
- Data source
- Business definition

This avoids different departments calculating the same KPI differently.

10.30 Reporting Architecture Review (Strategic Recommendations)

While writing this chapter, I identified several opportunities to elevate the reporting engine beyond traditional dashboards.

Recommendation 1 — Semantic KPI Layer

Instead of embedding formulas in charts, create a centralized KPI Catalog. Every dashboard, report, and API references the same KPI definitions. A change to a formula automatically updates every visualization.

Recommendation 2 — Snapshot Engine

At the end of each month, automatically capture a complete project snapshot:

- DT Status
- Acceptance Status
- Contractor Performance
- Province Performance

- KPI Values

This allows historical comparisons even if underlying operational data changes later.

Recommendation 3 — Executive Storyboards

In addition to dashboards, generate narrative reports.

Example:

July Summary

- DT completion increased by **8.3%** compared to June.
- CRA approvals improved in the North region.
- Two contractors exceeded SLA targets.
- Five provinces require immediate attention due to ICT backlog.

This combines charts with automatically generated management commentary.

Recommendation 4 — Decision Support Dashboard

Beyond showing numbers, display recommended actions.

Examples:

- "Reassign 42 Work Items from Contractor A to Contractor B."
- "Submit 18 Villages to ICT today."
- "Review 7 pending CPM conflicts."
- "Expedite CRA approvals in Region West."

This transforms reporting from descriptive analytics into prescriptive guidance.

Recommendation 5 — Enterprise Data Warehouse (Future)

As Budget, Battery, Power, AMP, and future modules are added, introduce a separate analytical database (Data Warehouse) optimized for reporting while keeping PostgreSQL focused on operational transactions.

End of Chapter 10

Chief Solution Architect Assessment

At this point, the platform architecture is **essentially complete from a business perspective**.

What remains are the engineering chapters:

- **Chapter 11 — Deployment & DevOps**
- **Chapter 12 — Testing & Quality Assurance**
- **Chapter 13 — Product Roadmap & Future Architecture**

One final observation after writing Chapters 1–10: your original idea started as a **Drive Test & Acceptance Tracker**, but the architecture we've converged on is much broader. It is now a **USO Operations Management Platform**, where Drive Test, Acceptance, Budget, Battery, Power, and future modules are all coordinated through a single operational ecosystem. That shift—from a tracking tool to an operations platform—is, in my view, the defining architectural evolution of the project.

Software Requirements Specification (SRS)

Chapter 11 — Deployment, Infrastructure & DevOps

Version: 1.0 Draft

11.1 Purpose

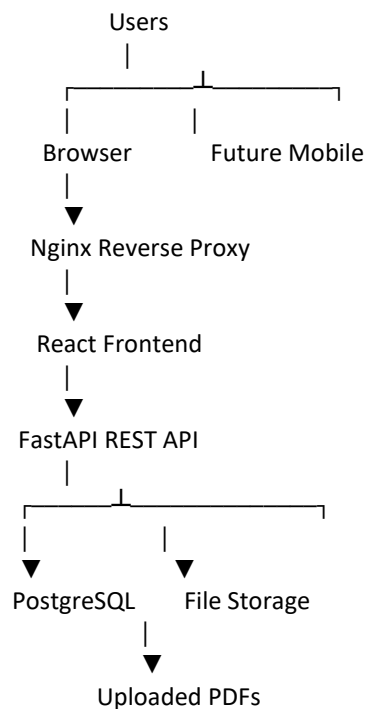
This chapter defines:

- Infrastructure
- Deployment strategy
- DevOps pipeline
- Environment management
- Monitoring

- Logging
- Backup
- Disaster Recovery
- Scalability

The objective is to provide a secure, maintainable, and production-ready platform.

11.2 High-Level Architecture



Future integrations:

- AMP
 - Budget Module
 - ERP
 - Power BI
 - Email Server
-

11.3 Technology Stack

Frontend

Component	Technology
Framework	React 19
Language	TypeScript
Build Tool	Vite
UI Library	Material UI
Charts	Apache ECharts
Tables	AG Grid Enterprise
Forms	React Hook Form
Validation	Zod

Backend

Component	Technology
Language	Python 3.12
Framework	FastAPI
ORM	SQLAlchemy
Validation	Pydantic
Authentication	JWT
API Docs	Swagger / OpenAPI

Database

Component	Technology
Primary Database	PostgreSQL
Migration	Alembic
Backup	pg_dump

Infrastructure

Component	Technology
Web Server	Nginx
Containerization	Docker

Component	Technology
Orchestration (Future)	Kubernetes
SSL	Let's Encrypt/Internal CA

11.4 Environment Strategy

Three environments shall exist.

Development

↓

Testing (UAT)

↓

Production

Each environment has:

- Independent database
- Independent configuration
- Independent storage
- Independent secrets

No shared resources between environments.

11.5 Configuration Management

Configuration must never be hardcoded.

Environment variables store:

Database URL

JWT Secret

SMTP

API Keys

Storage Path

Log Level

Example

.env.production

.env.testing

.env.local

11.6 Source Code Structure

Frontend

/src

/components

/pages

/services

/hooks

/layouts

/assets

/utils

Backend

/app

/api

/services

/models

/repositories

/core

/security

/workflows

/reports

/jobs

/tests

The structure separates business logic from infrastructure.

11.7 Branching Strategy

Git Flow:

main

↓

develop

↓

feature/*

↓

release/*

↓

hotfix/*

Rules:

- main always deployable
 - develop for integration
 - Features merged via Pull Requests
 - No direct commits to main
-

11.8 CI/CD Pipeline

Developer Push

↓

GitHub Actions



Run Tests



Lint



Build Frontend



Build Backend



Create Docker Images



Deploy to Test



Manual Approval



Deploy to Production

Every deployment is automated.

11.9 Docker Architecture

Containers:

Nginx

React

FastAPI

PostgreSQL

Redis (Future)

Worker (Future)

Each service runs independently.

11.10 Reverse Proxy

Nginx responsibilities:

- SSL termination
 - Static file serving
 - Reverse proxy
 - Compression (Gzip/Brotli)
 - Security headers
 - Rate limiting
-

11.11 File Storage

Uploaded files include:

- ICT PDFs
- CRA PDFs
- Import Templates
- Export Reports

Storage structure:

/storage

/cpm

/letters

/reports

/imports

/temp

Files are referenced in the database but stored on disk (or object storage in the future).

11.12 Scheduled Jobs

Background scheduler handles:

- Scheduled reports
- Notifications
- Backup
- Cleanup
- KPI recalculation
- Snapshot generation

Future: Celery + Redis or FastAPI Background Tasks.

11.13 Logging Strategy

Every application event generates structured logs.

Levels:

- INFO
- WARNING
- ERROR
- CRITICAL

Each log includes:

Timestamp

User

Module

Action

Duration

IP

Request ID

Logs should be JSON-formatted for easy aggregation.

11.14 Monitoring

Monitor:

- CPU
- Memory
- Disk
- Database
- API latency
- Failed requests
- Active users

Future tools:

- Prometheus
 - Grafana
-

11.15 Audit Logging

Separate from technical logs.

Every business action is recorded:

Who

Changed What

Old Value

New Value

When

Reason

Audit logs are immutable.

11.16 Backup Strategy

Database:

- Daily full backup
- Hourly incremental (future)

Files:

- Daily backup

Retention:

- Daily: 30 days
- Monthly: 12 months
- Yearly: 7 years

Backups are encrypted.

11.17 Disaster Recovery

Recovery objectives:

Metric	Target
RPO (Recovery Point Objective)	24 hours
RTO (Recovery Time Objective)	4 hours

Disaster recovery procedures must be documented and tested annually.

11.18 Performance Targets

Operation	Target
Login	<2 sec
Dashboard	<2 sec
Search	<1 sec
Record Save	<2 sec

Operation	Target
Import (5,000 rows)	<60 sec
Export Excel	<30 sec

These are non-functional requirements.

11.19 Scalability Strategy

Version 1:

Single server deployment.

Future:

Load Balancer

↓

Multiple FastAPI Instances

↓

Shared PostgreSQL

↓

Redis Cache

↓

Object Storage

The architecture supports horizontal scaling without redesign.

11.20 Security Operations

Production security includes:

- HTTPS only
- Strong password policy

- JWT expiration
 - Automatic logout
 - Account lockout after repeated failures
 - CSRF protection where applicable
 - SQL injection prevention (ORM)
 - XSS protection
 - Secure HTTP headers
-

11.21 Email Service

The application sends:

- Assignment notifications
- Approval requests
- Scheduled reports
- Password reset
- System alerts

Emails are asynchronous and logged.

11.22 Version Management

Every release receives:

Version

Release Date

Change Log

Database Migration

Rollback Plan

Semantic versioning:

1.0.0

1.1.0

1.2.0

11.23 Deployment Checklist

Before Production:

- All tests passed
- Database migration validated
- Backups verified
- Security review completed
- Performance baseline measured
- Rollback package prepared
- Monitoring enabled

Deployment is not complete until post-deployment validation succeeds.

11.24 Infrastructure Documentation

Maintain documentation for:

- Network topology
- Server inventory
- Environment variables
- Deployment steps
- Backup procedures
- Recovery procedures
- External integrations

Documentation must be version-controlled.

11.25 DevOps Recommendations (Architect Review)

During the infrastructure design, I identified several enhancements that would significantly improve long-term maintainability.

Recommendation 1 — Event Bus (Future)

Introduce an internal event bus.

Examples:

- Work Item Assigned
- DT Validated
- ICT Approved
- CRA Approved
- CPM Imported

Instead of modules calling each other directly, they publish events. This reduces coupling and makes future integrations easier.

Recommendation 2 — Feature Flags

Deploy unfinished features safely.

Example:

Battery Module

Enabled: False

The code can exist in production without being visible until ready.

Recommendation 3 — Centralized Error Handling

Create a single error middleware that:

- Logs the exception
- Generates a Request ID
- Returns a user-friendly message
- Notifies administrators for critical failures

This ensures consistent error responses.

Recommendation 4 — Health Check Endpoints

Expose endpoints:

/health

/ready

/live

Used by monitoring systems and future Kubernetes deployments to verify application status.

Recommendation 5 — Infrastructure as Code

Although Version 1 may run on a single VPS, document the infrastructure using Terraform or Ansible in the future.

This enables reproducible deployments and simplifies disaster recovery.

Recommendation 6 — Blue/Green Deployment

For future production upgrades, deploy the new version alongside the current one, validate it, and then switch traffic. This minimizes downtime and provides an easy rollback path.

11.26 Long-Term Platform Vision

By the time all planned modules are implemented, the platform will include:

- Drive Test Management
- Acceptance Management
- Budget Management
- Battery Tracking
- Power Tracking
- CPM Synchronization
- Reporting & BI
- Workflow Automation
- Notification Center
- Action Center

- Executive Cockpit
- AI Decision Support (Future)

At that point, the system evolves into a **USO Digital Operations Platform** capable of supporting the complete operational lifecycle of the project.

End of Chapter 11

Chief Solution Architect Assessment

With Chapters **1 through 11**, the system is now fully specified from a functional, architectural, and infrastructure perspective. A development team could begin implementation using these documents as the primary blueprint.

The remaining chapters are about **quality and longevity** rather than new functionality:

- **Chapter 12 — Testing, QA & Validation** will define how we prove the platform works correctly, securely, and performs under load before each release.
- **Chapter 13 — Product Roadmap & Future Architecture** will capture the evolution toward AI-assisted operations, advanced integrations, and enterprise-scale growth over the next several years.

At this stage, I would classify the project as **production-ready by design**, with a clear path from an initial single-server deployment to a scalable enterprise platform without requiring architectural rework.

Software Requirements Specification (SRS)

Chapter 12 — Testing, Quality Assurance & Validation

Version: 1.0 Draft

12.1 Purpose

The Quality Assurance (QA) strategy ensures that the USO Operations Platform is:

- Correct
- Reliable
- Secure
- Performant
- Maintainable
- Regression-free

Testing shall be performed throughout the software lifecycle, not only before production deployment.

12.2 Testing Strategy

The project follows the **Testing Pyramid**.

UI Tests
Integration Tests
Service Tests
Unit Tests

Approximate distribution:

- Unit Tests: 70%
- Integration Tests: 20%
- End-to-End Tests: 10%

12.3 Testing Levels

The platform shall be tested at multiple levels.

Level 1

Unit Testing

Validate individual business functions.

Example:

Calculate DT Progress

Validate Work Item Status

Generate KPI

Level 2

Integration Testing

Validate interaction between modules.

Example

Assignment

↓

Health Check

↓

Drive Test

↓

Acceptance

Level 3

System Testing

Validate complete workflows.

Example

Import CPM

↓

Assign Contractor

↓

Health Check

↓

Drive Test



Acceptance



Report

Level 4

User Acceptance Testing (UAT)

Performed by:

- Project Manager
- Coordinators
- Contractors
- Regional Managers

Only after successful UAT may the release proceed to production.

12.4 Functional Testing

Every business requirement shall have at least one corresponding test case.

Examples:

Feature	Test
Assignment	Assign Work Item
DT	Validate DT
Acceptance	ICT Approval
Acceptance	CRA Approval
Reporting	Dashboard KPI
Security	Role Permission

Requirement traceability shall be maintained.

12.5 Workflow Validation

The complete business lifecycle must be validated.

CPM Import



Assignment



Health Check



DT Submission



Coordinator Validation



PM Approval



Acceptance



Completed

Each transition shall verify:

- Business rules
- Permissions
- Audit logging
- Notifications
- Dashboard updates

12.6 Business Rule Testing

Special attention shall be given to business rules.

Examples:

- ✓ Work Item cannot be DT Completed before Assignment.
 - ✓ PM Approval cannot occur before Coordinator Approval.
 - ✓ Village Approval requires valid ICT/CRA status.
 - ✓ Deleted Work Items remain recoverable (Soft Delete).
 - ✓ CPM conflicts require PM decision.
-

12.7 Security Testing

The following security aspects shall be verified.

Authentication

Authorization

JWT Validation

Password Policy

Session Timeout

Role Isolation

Audit Logging

Permission Escalation

Direct URL Access

SQL Injection

Cross Site Scripting (XSS)

Cross Site Request Forgery (CSRF)

12.8 Performance Testing

Performance testing shall measure:

Dashboard Load

Search

Filtering

Bulk Import

Bulk Export

Report Generation

Database Queries

API Latency

Acceptance Criteria

Operation	Target
Dashboard	<2 sec
Search	<1 sec
Excel Export	<30 sec
PDF Report	<20 sec
Import 5000 Records	<60 sec

12.9 Load Testing

Expected Version 1

Concurrent Users

40

Future Target

200+

Load testing shall simulate:

Multiple coordinators

Multiple contractors

Report generation

Concurrent imports

Concurrent dashboard refresh

12.10 Stress Testing

The system shall continue operating under abnormal conditions.

Examples:

- Massive CPM import
- Thousands of notifications
- Large PDF generation
- Database slowdown
- Simultaneous report exports

Expected behavior:

Graceful degradation without data corruption.

12.11 Recovery Testing

Verify recovery after failures.

Examples:

Server Restart

Database Restart

Power Failure

Unexpected Shutdown

Recovery shall preserve committed transactions.

12.12 Database Testing

Validate:

Primary Keys

Foreign Keys

Indexes

Cascade Rules

Soft Delete

History

Audit Tables

Migration Scripts

No orphan records shall exist.

12.13 API Testing

Every REST endpoint shall be validated.

Verify:

Status Codes

Response Structure

Validation

Authorization

Rate Limits

Pagination

Filtering

Sorting

Error Handling

Swagger Documentation

12.14 UI Testing

Verify:

Responsive Layout

Tables

Charts

Dark Mode

Dialogs

Timeline

Dashboard

Notifications

Action Center

Supported browsers:

Chrome

Edge

Firefox

12.15 Regression Testing

Every release shall execute automated regression tests.

Critical workflows:

Login

Assignment

Health Check

Drive Test

Acceptance

Reporting

User Management

Regression failures block production deployment.

12.16 Import Validation

One of the most critical modules.

Every import shall verify:

File Format

Required Columns

Duplicate Records

Invalid Values

Business Rules

Unknown Sites

Unknown Villages

Technology Conflicts

Import Preview

Only validated records may be committed.

12.17 Reporting Validation

Every KPI must be verified.

Example

Excel

↓

Platform

↓

Results must match

Dashboard numbers shall be cross-checked against SQL queries and historical Excel calculations during migration.

12.18 User Acceptance Testing (UAT)

Representative users:

Project Manager

Coordinator

Contractor

Regional Manager

Viewer

Admin

Each role executes predefined scenarios.

Sign-off is mandatory.

12.19 Test Data Management

Three datasets shall be maintained.

Development

Synthetic Data

Testing

Anonymized Production-like Data

Production

Real Data

No production database shall be used for development.

12.20 Defect Management

Each defect includes:

ID

Summary

Severity

Priority

Module

Steps to Reproduce

Expected Result

Actual Result

Owner

Status

Resolution

Severity Levels:

Critical

High

Medium

Low

12.21 Release Acceptance Criteria

A release is approved only if:

- All Critical defects closed
 - All High defects closed or formally accepted
 - Regression tests passed
 - Performance targets achieved
 - Security tests passed
 - UAT approved
 - Deployment checklist completed
-

12.22 Traceability Matrix

Every requirement shall be linked to:

Requirement



Module



API



Database Entity



Test Case



UAT Scenario



Release

This ensures complete traceability throughout the project lifecycle.

12.23 Automation Strategy

Automated testing should include:

Backend

- Unit Tests (Pytest)
- API Tests
- Business Rules

Frontend

- Component Tests
- UI Tests
- End-to-End Tests (Playwright)

CI/CD

Every Pull Request automatically runs:

- Unit Tests
- Integration Tests
- Static Analysis
- Security Scan

Deployment is blocked if any mandatory test fails.

12.24 Test Documentation

Maintain the following documents:

- Test Plan
- Test Cases
- Test Scripts
- Regression Suite
- UAT Scripts
- Defect Log
- Test Summary Report

All documents shall be version-controlled.

12.25 Operational Readiness Review (ORR)

Before go-live, conduct an Operational Readiness Review.

Checklist includes:

- Infrastructure validated

- Monitoring enabled
- Backups tested
- Security review completed
- User accounts created
- Initial data migrated
- Reports validated
- Support procedures documented
- Rollback plan verified

Go-live requires formal approval from the Project Manager.

12.26 Quality Metrics

Track quality over time using measurable indicators.

Metric	Target
Test Coverage	≥85%
Critical Defects	0
High Defects	0
Regression Pass Rate	100%
API Availability	≥99.9%
Import Success Rate	≥99%
Report Accuracy	100%
Mean Time to Resolve (MTTR)	<3 Business Days

These metrics should be reviewed after every release.

12.27 Production Validation (Post Go-Live)

Testing does not end after deployment.

Within the first week of production:

- Validate dashboard totals
- Compare KPIs with legacy Excel reports
- Confirm contractor assignments

- Verify ICT/CRA approval updates
- Review audit logs
- Check scheduled jobs
- Validate notification delivery
- Confirm backup execution

A production validation report shall be produced.

12.28 Quality Improvements (Architect Recommendations)

While designing the QA strategy, I identified several enhancements that would greatly improve long-term reliability.

Recommendation 1 — Business Rule Test Suite

Create a dedicated automated suite focused only on business rules.

Example:

- Cannot approve DT before Coordinator Validation.
- Cannot submit Acceptance without DT completion.
- CPM conflicts must trigger PM review.

These tests become the "contract" protecting your workflow from future regressions.

Recommendation 2 — Golden Dataset

Maintain a fixed reference dataset containing a representative USO project.

Every release should run all reports against this dataset and compare the results with expected values. If even one KPI changes unexpectedly, the build fails.

This guarantees reporting consistency over time.

Recommendation 3 — Synthetic Production Simulator

Generate large synthetic datasets (e.g., 50,000 Sites, 250,000 Villages) to stress-test imports, dashboards, filters, and reports before introducing future modules like Budget, Battery, and Power.

Recommendation 4 — Audit Verification Tests

Automate verification that every significant business action creates:

- An Audit Log
- A History Record
- A Notification (if applicable)
- An Action Center item (if applicable)

This ensures accountability is never lost as the system evolves.

Recommendation 5 — Release Certification

Introduce a formal **Release Certification Report** for each production version.

The report should summarize:

- Test execution statistics
- Defect status
- Performance benchmarks
- Security scan results
- UAT approval
- Deployment readiness

This creates an auditable quality record for every release.

End of Chapter 12

Chief Solution Architect Assessment

With Chapter 12 complete, the project now includes a comprehensive quality strategy covering functional correctness, workflow validation, security, performance, reporting accuracy, and

operational readiness. At this stage, the platform is specified not only for **how it should be built**, but also **how its correctness will be demonstrated and maintained**.

Software Requirements Specification (SRS)

Chapter 13 — Product Roadmap & Future Architecture

Version: 1.0 Draft

13.1 Purpose

This chapter defines the long-term strategic vision of the USO Operations Platform beyond Version 1.

It identifies:

- Product evolution
- Future modules
- Enterprise integrations
- AI capabilities
- Scalability roadmap
- Technology evolution
- Organizational adoption

The objective is to ensure that architectural decisions made today will continue to support future business growth without requiring fundamental redesign.

13.2 Product Vision

The current Version 1 focuses on:

- Drive Test Management
- ICT Acceptance
- CRA Acceptance

- Reporting

The long-term vision is significantly broader.

The platform shall become the **single operational system** for managing the complete lifecycle of every USO site—from assignment to final project closure.

13.3 Product Evolution

Phase 1 — Operational Foundation (Version 1)

Core capabilities:

- Login & Security
- CPM Import
- Assignment
- Health Check
- Drive Test
- Acceptance
- Reporting
- Notifications
- Action Center
- Executive Dashboard

Status: **Current Project**

Phase 2 — Operational Expansion (Version 2)

New modules:

- Budget Management
- Purchase Order Tracking
- Invoice Processing
- Depreciation Management
- Contractor Performance
- Financial Reporting

This phase consolidates the budget application you previously developed into the same platform.

Phase 3 — Asset Management (Version 3)

Additional modules:

- Battery Status
- Temporary Power
- Permanent Power
- AMP Integration
- Equipment Tracking
- Asset Health

These modules are associated with the **Site** entity rather than the **Work Item**, enabling infrastructure lifecycle management.

Phase 4 — Executive Intelligence (Version 4)

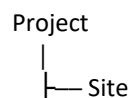
Advanced capabilities:

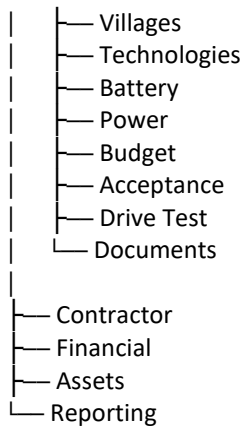
- Predictive Dashboards
- AI Assistant
- Forecasting
- Resource Planning
- Delay Prediction
- Executive Cockpit

This phase transforms the platform into a strategic decision-support system.

13.4 Long-Term Domain Model

By Version 4, the domain expands significantly.





This evolution maintains backward compatibility with Version 1.

13.5 Enterprise Integration Roadmap

Future integrations include:

System	Purpose
CPM	Master Data
AMP	Battery & Power
Budget Tool	Financial Management
ERP	Purchase Orders & Invoices
SMTP	Email Notifications
Power BI	Executive Analytics
Active Directory Single Sign-On (Future)	

All integrations should use standardized REST APIs where possible.

13.6 AI-Assisted Operations

Artificial Intelligence should augment—not replace—human decision-making.

Planned AI capabilities include:

Intelligent Assignment Suggestions

Recommend the most suitable contractor based on:

- Region
 - Current workload
 - Historical performance
 - SLA compliance
-

Acceptance Risk Prediction

Predict which sites are likely to be rejected based on:

- Technology
 - Historical trends
 - Region
 - Contractor
 - Similar past cases
-

Executive Q&A

Natural-language queries such as:

- "Show me all blocked sites in the North region."
- "Which contractor has the highest rejection rate this month?"
- "How many villages are waiting for ICT approval?"

The system should translate these into secure database queries.

Root Cause Analysis

Automatically identify recurring causes of delays or rejections and present actionable insights.

13.7 Advanced Analytics

Future dashboards may include:

- Forecasted DT completion date
- Acceptance completion forecast
- Monthly backlog trends
- Regional heat maps
- Contractor scorecards
- Technology-specific performance
- SLA compliance
- Budget burn rate
- Battery rollout progress
- Power readiness

These analytics support proactive project management.

13.8 Intelligent Action Center

The Action Center evolves from a notification list into a decision-support engine.

Examples:

- "15 sites are ready for ICT submission."
- "3 CPM conflicts require PM approval."
- "Contractor A has exceeded the DT SLA."
- "CRA Region West has 45 pending villages."

Each action includes:

- Priority
 - Owner
 - Due Date
 - Suggested Resolution
-

13.9 Workflow Automation

Routine tasks should become automated.

Examples:

- Automatic assignment emails

- Reminder notifications
- Scheduled reports
- SLA breach alerts
- Approval escalations
- CPM change detection

The goal is to minimize manual coordination.

13.10 Knowledge Base

The platform should maintain organizational knowledge, including:

- Standard Operating Procedures (SOPs)
- User Guides
- Acceptance Rules
- Common Issues
- Resolution Procedures
- Lessons Learned

This reduces dependency on individual expertise.

13.11 Executive Cockpit

A dedicated dashboard for senior management should provide:

- Overall Project Progress
- Regional Performance
- Contractor Performance
- Financial Status
- Acceptance Status
- Risk Indicators
- Forecasted Completion
- Executive Alerts

The interface should prioritize clarity and strategic insights over operational details.

13.12 Mobile Readiness

While Version 1 targets desktop users, the architecture should support future mobile applications.

Potential mobile features:

- Contractor status updates
- Health Check confirmation
- Push notifications
- Executive dashboards
- Approval workflows

Business logic should remain centralized in the backend APIs.

13.13 Scalability Roadmap

The platform should evolve without architectural redesign.

Phase	Target
Version 1	Single Server
Version 2	Docker Deployment
Version 3	Load Balancer + Multiple API Instances
Version 4	Kubernetes + High Availability

The architecture already supports horizontal scaling.

13.14 Data Growth Strategy

Expected long-term growth:

- Tens of thousands of Sites
- Hundreds of thousands of Villages
- Millions of Approval Records
- Millions of Audit Records
- Thousands of Documents

Strategies include:

- Database indexing
- Archiving
- Partitioning
- Read replicas (future)

13.15 Product Governance

A governance process should oversee future changes.

Responsibilities include:

- Requirement approval
- Architecture review
- Release planning
- Security review
- Data governance
- KPI ownership

A Product Steering Committee is recommended for major enhancements.

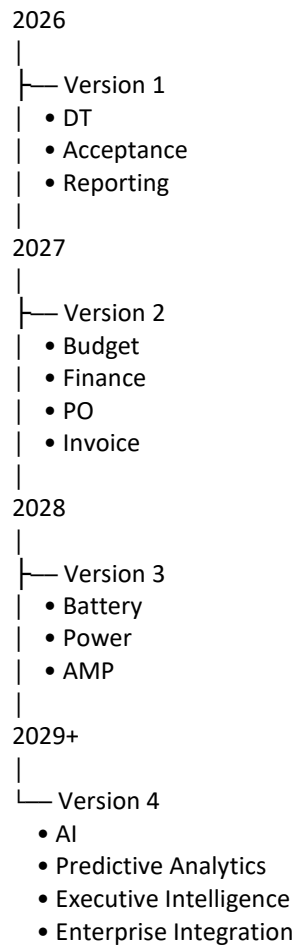
13.16 Innovation Opportunities

Potential future innovations include:

- GIS map integration
- Interactive network coverage maps
- QR codes for Site verification
- Digital approval workflows
- OCR for letter processing
- AI-assisted document classification
- Voice-based reporting
- Predictive maintenance for infrastructure

These should be evaluated based on business value.

13.17 Product Roadmap Timeline



The roadmap should remain flexible and driven by business priorities.

13.18 Architectural Principles

All future development should adhere to the following principles:

- Single Source of Truth
- API-First Design
- Modular Architecture
- Event-Driven Integration
- Secure by Default
- Scalability by Design
- Auditability

- Backward Compatibility
- User-Centered Design

These principles ensure consistency across all future releases.

13.19 Success Vision

The ultimate success of the platform is not measured by the number of features implemented, but by its adoption as the primary operational system for the USO project.

Success indicators include:

- Complete elimination of operational Excel trackers.
 - Real-time visibility into project execution.
 - Improved collaboration across teams.
 - Faster decision-making.
 - Reduced operational delays.
 - Reliable executive reporting.
 - Sustainable architecture for future growth.
-

13.20 Chief Solution Architect Recommendations

After reviewing the entire solution, I have several strategic recommendations:

Recommendation 1 — Product Name

Move beyond the working title "Acceptance Tool."

Recommended names include:

- **USO Operations Platform (Recommended)**
- USO Project Hub
- USO Digital Operations
- Telecom Operations Platform

A broader name reflects the platform's long-term scope.

Recommendation 2 — Product Ownership

Appoint a dedicated Product Owner responsible for prioritizing enhancements, maintaining the roadmap, and ensuring alignment with business objectives.

Recommendation 3 — API-First Mindset

Every new module should expose standardized APIs, enabling future integrations with ERP systems, analytics platforms, and mobile applications without rework.

Recommendation 4 — Business KPI Governance

Maintain a centralized KPI catalog where every metric has:

- Business definition
- Formula
- Data source
- Owner
- Update frequency

This prevents conflicting interpretations across reports.

Recommendation 5 — AI as an Assistant

Introduce AI gradually to support decision-making rather than automate approvals. Human oversight should remain mandatory for critical project decisions.

Recommendation 6 — Documentation Discipline

Treat this SRS as a living document. Every major release should update:

- Business Process
- Domain Model
- Database Schema

- APIs
- UI
- Security
- Roadmap

This ensures the documentation evolves with the platform.

End of Chapter 13

Final Solution Architect Assessment

With Chapters **1 through 13**, the project now has a complete enterprise-grade blueprint encompassing business strategy, functional requirements, domain modeling, database architecture, module decomposition, security, APIs, user experience, reporting, deployment, quality assurance, and long-term product evolution.

From an architectural standpoint, this is no longer a specification for a "Drive Test Tracker." It defines a scalable **USO Operations Platform** capable of supporting the full operational lifecycle of telecom infrastructure projects.

The documentation is sufficiently comprehensive to guide implementation, support governance reviews, onboard new development teams, and serve as the foundation for future enhancements over the coming years. By maintaining disciplined version control and adhering to the architectural principles defined throughout these chapters, the platform can evolve organically without requiring disruptive redesigns, ensuring both technical sustainability and long-term business value.